



ESCUELA POLITÉCNICA NACIONAL



ESCUELA DE FORMACIÓN DE TECNÓLOGOS

BASES DE DATOS

PROYECTO FINAL

Modernización de una Empresa de Calzado Nacional – Ecuador

Integrantes

- José Buñay
- Kevin Chillagana
- Francisco Lanche

Fecha de entrega: 23/07/2026

- **INTRODUCCIÓN**

1.1. Descripción del problema

La empresa que sirve de caso de estudio es un negocio ecuatoriano dedicado a la fabricación y comercialización de calzado nacional, con presencia en varias provincias a través de vendedores y puntos de venta propios. Hasta antes de este proyecto, el control de inventario, las compras a proveedores y el registro de ventas se llevaba en hojas de cálculo independientes que cada sucursal manejaba por su cuenta. Esta forma de trabajo generaba varios problemas concretos: el stock que se mostraba en una hoja no coincidía con el stock real de bodega, algunos productos se vendían sin verificar existencias, no había forma rápida de saber qué vendedor generaba más ingresos o qué proveedor entregaba los productos más solicitados, y las devoluciones se anotaban de forma manual sin quedar vinculadas a la venta original. A esto se sumaba la falta de un registro de quién modificaba precios o datos de clientes, lo que dificultaba detectar errores o accesos indebidos. En conjunto, esta situación afectaba la toma de decisiones comerciales, retrasaba el cierre de caja y aumentaba el riesgo de pérdidas por descuadres de inventario.

1.2. Objetivo general

Diseñar e implementar, en un motor de base de datos Oracle, un sistema de información que centralice y automatice la gestión de clientes, productos, proveedores, empleados, ventas, devoluciones, promociones e inventario de la empresa de calzado, garantizando la integridad de los datos y dejando trazabilidad de las operaciones más sensibles.

1.3. Objetivos específicos

- Levantar un modelo entidad-relación normalizado que represente correctamente los módulos de clientes, productos, proveedores, empleados, ventas, compras, devoluciones, promociones, inventario y auditoría.
- Implementar el esquema físico en Oracle con los tipos de dato, restricciones e integridad referencial que correspondan a cada atributo y a cada relación entre tablas.

- Automatizar los cálculos recurrentes del negocio (IVA, descuentos, comisiones, antigüedad de clientes) mediante funciones reutilizables desde cualquier consulta o procedimiento.
- Encapsular las operaciones críticas (registrar venta, registrar devolución, actualizar inventario, aplicar promoción) en procedimientos almacenados que validen las reglas de negocio antes de modificar los datos.
- Mantener el stock siempre consistente usando triggers que descuenten o reintegren unidades automáticamente según se registren ventas o devoluciones.
- Registrar de forma automática los cambios sobre datos sensibles (clientes, precios, accesos al esquema) en una tabla de auditoría.
- Definir roles y usuarios con el privilegio mínimo necesario según la función de cada perfil dentro de la empresa.
- Establecer una estrategia de respaldo que permita recuperar el esquema completo ante una falla o pérdida de información.

1.4. Alcance del sistema

El sistema cubre el ciclo completo de la operación comercial de una sucursal de calzado: registro y consulta de clientes, catálogo de productos con su categoría y proveedor, compras a proveedores, ventas al detalle o al por mayor con aplicación opcional de promociones, devoluciones asociadas a una venta específica, control de inventario por movimientos de entrada y salida, y generación de reportes analíticos mediante vistas. Incluye además la capa de seguridad (roles y usuarios) y la capa de auditoría (registro de cambios y accesos). Queda fuera del alcance la integración con hardware de punto de venta (impresoras fiscales, lectores de código de barras), la facturación electrónica ante el SRI y la sincronización en tiempo real entre sucursales, aspectos que se plantean como trabajo futuro y se mencionan en la sección de recomendaciones.

- Un cliente no puede registrarse dos veces con la misma cédula ni con el mismo correo electrónico.
- El stock de un producto nunca puede quedar en un valor negativo; toda salida se valida contra la existencia disponible antes de confirmarse.

- Toda venta debe incluir al menos un producto y debe clasificarse como venta "Mayor" o "Menor".
- Una devolución solo puede registrarse si el producto pertenece efectivamente a la venta indicada, y la cantidad devuelta no puede superar la cantidad que quedó disponible después de devoluciones anteriores sobre la misma venta.
- Un proveedor debe clasificarse como "Nacional" o "Importado", y su número de teléfono debe tener exactamente diez dígitos.
- El IVA se calcula de forma uniforme como el 15% del subtotal, una vez aplicado el descuento correspondiente.
- El descuento de una venta depende del tipo de promoción vigente: puede ser un porcentaje sobre el subtotal o un valor fijo, según cómo esté configurada la promoción.
- Un cliente se considera "frecuente" cuando acumula más de cinco compras o un monto total igual o mayor a 500 dólares.
- Todo cambio en el precio de un producto y todo cambio en los datos de un cliente debe quedar registrado en la tabla de auditoría, junto con el valor anterior y el nuevo.

Cada inicio de sesión sobre el esquema de base de datos se registra automáticamente, sin intervención del usuario.

2.2. DISEÑO FÍSICO

- **Script de creación de tablas**

[Enlace al script.](#)

- **Restricciones aplicadas**

NOT NULL: aplicado a todo campo que es obligatorio para que el registro tenga sentido de negocio (nombres, cédulas, precios, cantidades, fechas de eventos, etc.). Los únicos campos nullable son los que representan datos opcionales por diseño: promoción en venta (no toda venta usa promoción), motivo en devolución/condición en promoción (texto libre opcional), e id_cliente en queja (para permitir quejas anónimas).

UNIQUE: aplicado a identificadores de negocio que no deben duplicarse aunque no sean la PK: cédula y correo en cliente y empleado, nombre en proveedor/producto/categoría/promoción, y teléfono/correo en proveedor. Esto

evita registros duplicados de la misma persona o entidad aunque tengan un ID distinto.

CHECK: usado en dos formas — (1) listas cerradas de valores válidos, por ejemplo tipo_proveedor IN ('Nacional','Importado'), tipo_venta IN ('Mayor','Menor'), tipo_movimiento IN ('Entrada','Salida'), tipo IN ('Reclamo','Sugerencia'); y (2) validación de rangos numéricos no negativos, por ejemplo precio ≥ 0 , stock_actual ≥ 0 , cantidad > 0 (estricta, ya que un movimiento de cantidad 0 no tiene sentido). Estas reglas quedan garantizadas a nivel de motor, independientemente de qué aplicación o usuario inserte los datos.

3. CARGA DE DATOS

3.1. Script de inserción de datos de prueba

[Enlace al script.](#)

3.2. Volumen de datos cargado por tabla

Tabla	Registros cargados
proveedor	15
empleado	15
cliente	35
promocion	15
categoria	8
producto	25
inventario	19
venta	55
detalle_venta	144
devolucion	25
compra	15
detalle_compra	44
auditoria	0
queja	0

4. CONSULTAS

4.1. Listado de clientes

Devuelve el listado general de clientes con sus datos de contacto (nombre, apellido, teléfono, correo) y la provincia donde están registrados.

```

SELECT
  id_cliente,
  CONCAT(nombre, ' ', apellido) as nombre_cliente,
  telefono,
  correo,
  provincia
FROM cliente
ORDER BY nombre DESC;

```

Se han recuperado todas las filas: 35 en 0.028 segundos

	ID_CLIENTE	NOMBRE_CLIENTE	TELEFONO	CORREO
1	25	Xavier Aguirre	0968812137	xavier.aguirre24@correo.com
2	33	Wilson Tapia	0985146293	wilson.tapia32@correo.com
3	14	Veronica Villacis	0920117988	veronica.villacis13@correo.com
4	30	Tatiana Merino	0984045292	tatiana.merino29@correo.com
5	22	Silvia Loja	0952169044	silvia.loja21@correo.com
6	7	Sebastian Rivera	0926240908	sebastian.rivera6@correo.com
7	27	Ruben Cadena	0930863865	ruben.cadena26@correo.com
8	3	Roberto Castro	0951273847	roberto.castro2@correo.com
9	11	Ricardo Guerrero	0998447167	ricardo.guerrero10@correo.com

4.2. Productos disponibles

Muestra los productos disponibles junto con su tipo, precio de venta y el stock actual, útil como consulta rápida de catálogo.

```

SELECT p.id_producto,
  p.nombre AS nombre_producto,
  pr.tipo_proveedor AS tipo_producto,
  p.precio,
  p.stock_actual AS stock_actual
FROM producto p
JOIN proveedor pr ON p.id_proveedor = pr.id_proveedor
ORDER BY p.nombre;

```

Se han recuperado todas las filas: 25 en 0.023 segundos

	ID_PRODUCTO	NOMBRE_PRODUCTO	TIPO_PRODUCTO	PRECIO	STOCK_ACTUAL
1	20	Bota Alta Cuero Invierno	Nacional	55	26
2	19	Bota de Lluvia Impermeable	Nacional	35	30
3	21	Bota de Trabajo Resistente	Nacional	60	28
4	11	Bota Outdoor Impermeable X	Importado	120	18
5	12	Bota Outdoor Montana	Importado	115	19
6	10	Bota Outdoor Trekking Pro	Importado	110	20
7	15	Sandalia Casual Verano	Nacional	24.5	52
8	14	Sandalia Deportiva Trail	Importado	38	40
9	13	Sandalia Playera Comfort	Nacional	22.99	50

4.3. Ventas por fecha

Lista las ventas junto con la fecha en que se realizaron, el cliente asociado y el total facturado.

```
SELECT
  v.id_venta,
  v.fecha_venta,
  CONCAT(c.nombre, ' ', c.apellido) as nombre_cliente,
  v.total as total_venta
FROM venta v
JOIN cliente c ON c.id_cliente = v.id_cliente;
```

Se han recuperado todas las filas: 55 en 0.030 segundos

	ID_VENTA	FECHA_VENTA	NOMBRE_CLIENTE	TOTAL_VENTA
1	1	27/05/26	Gabriela Leon	4218.2
2	2	15/03/26	Daniela Morales	610.59
3	3	18/01/26	Cristina Paredes	84.77
4	4	16/02/26	Tatiana Merino	381.23
5	5	24/05/26	Fernanda Ponce	1438.19
6	6	27/03/26	Silvia Loja	2881.9
7	7	18/07/26	Bolivar Yanez	931.17
8	8	26/06/26	Roberto Castro	1044.2
9	9	04/04/26	Silvia Loja	162.39

4.4. Proveedores registrados

Muestra el listado de proveedores registrados con su país de origen, tipo (Nacional/Importado) y teléfono de contacto.

```
SELECT
  id_proveedor,
  nombre AS proveedor,
  pais,
  tipo_proveedor,
  telefono
FROM proveedor;
```

Se han recuperado todas las filas: 15 en 0.013 segundos

	ID_PROV...	PROVEEDOR	PAIS	TIPO_PROVEEDOR	TELEFONO
1	1	Calzado Gamos S.A.	Ecuador	Nacional	0987001001
2	2	Venus Calzado Cia Ltda	Ecuador	Nacional	0987001002
3	3	Buestan Calzado	Ecuador	Nacional	0987001003
4	4	Guangzhou Footwear Export	China	Importado	0987001004
5	5	Grupo Calzado del Pacifico	Colombia	Importado	0987001005
6	6	Industrias del Calzado Ambato	Ecuador	Nacional	0987001006
7	7	Nike Trading LLC	Estados Unidos	Importado	0987001007
8	8	Adidas Distribuidora Andina	Alemania	Importado	0987001008
9	9	Calzado Cuenca Export	Ecuador	Nacional	0987001009

4.5. Empleados por rol

Agrupar a los empleados según su rol dentro del sistema, mostrando también el cargo que ocupan.

```
SELECT
    id_empleado,
    nombre || ' ' || apellido AS nombre_empleado,
    cargo,
    rol
FROM empleado;
```

Se han recuperado todas las filas: 15 en 0.027 segundos

	ID_EMPLEADO	NOMBRE_EMPLEADO	CARGO	ROL
1	1	Carlos Perez	Cajero	Administracion
2	2	Maria Gomez	Supervisor de Ventas	Ventas
3	3	Juan Rodriguez	Gerente de Sucursal	Inventario
4	4	Ana Torres	Bodeguero	Ventas
5	5	Luis Vasquez	Vendedor	Administracion
6	6	Sofia Chavez	Contador	Inventario
7	7	Pedro Suarez	Contador	Contabilidad
8	8	Lucia Molina	Vendedor	Gerencia
9	9	Diego Cevallos	Bodeguero	Gerencia

4.6. Clientes con sus compras (JOIN)

Combina cliente y venta para mostrar, por cada cliente, las compras que ha realizado, su fecha y el monto total.

```
SELECT
    CONCAT(c.nombre, ' ', c.apellido) as nombre_cliente,
    v.fecha_venta,
    v.total as total_venta,
    v.tipo_venta
FROM cliente c
JOIN venta v ON v.id_cliente = c.id_cliente;
```

Se han recuperado todas las filas: 55 en 0.044 segundos

	NOMBRE_CLIENTE	FECHA_VENTA	TOTAL_VENTA	TIPO_VENTA
1	Gabriela Leon	27/05/26	4218.2	Mayor
2	Daniela Morales	15/03/26	610.59	Menor
3	Cristina Paredes	18/01/26	84.77	Menor
4	Tatiana Merino	16/02/26	381.23	Menor
5	Fernanda Ponce	24/05/26	1438.19	Mayor
6	Silvia Loja	27/03/26	2881.9	Mayor
7	Bolivar Yanez	18/07/26	931.17	Menor
8	Roberto Castro	26/06/26	1044.2	Menor
9	Silvia Loja	04/04/26	162.39	Menor

4.7. Ventas con vendedor (JOIN)

Combina venta, cliente y empleado para identificar qué vendedor (empleado) atendió cada venta.

```
SELECT
    v.id_venta,
    CONCAT(c.nombre, ' ', c.apellido) as nombre_cliente,
    CONCAT(e.nombre, ' ', e.apellido) as nombre_empleado,
    v.fecha_venta,
    v.total as total_venta
FROM venta v
JOIN cliente c ON c.id_cliente = v.id_cliente
JOIN empleado e ON e.id_empleado = v.id_empleado;
```

Se han recuperado todas las filas: 55 en 0.026 segundos

	ID_VENTA	NOMBRE_CLIENTE	NOMBRE_EMPLEADO	FECHA_VENTA	TOTAL_VENTA
1	1	Gabriela Leon	Jorge Salazar	27/05/26	4218.2
2	2	Daniela Morales	Ana Torres	15/03/26	610.59
3	3	Cristina Paredes	Valentina Freire	18/01/26	84.77
4	4	Tatiana Merino	Maria Gomez	16/02/26	381.23
5	5	Fernanda Ponce	Pedro Suarez	24/05/26	1438.19
6	6	Silvia Loja	Fernando Ortiz	27/03/26	2881.9
7	7	Bolivar Yanez	Lucia Molina	18/07/26	931.17
8	8	Roberto Castro	Sofia Chavez	26/06/26	1044.2
9	9	Silvia Loja	Paola Vega	04/04/26	162.39

4.8. Detalle de productos vendidos (JOIN)

Combina venta, detalle_venta y producto para mostrar línea por línea qué productos se vendieron en cada venta, con cantidad y subtotal.

```
SELECT
    v.id_venta,
    p.nombre AS Producto,
    dv.cantidad as Cantidad,
    dv.precio_unitario as PrecioUnitario,
    dv.subtotal AS Subtotal
FROM venta v
JOIN detalle_venta dv ON dv.id_venta=v.id_venta
JOIN producto p ON p.id_producto=dv.id_producto;
```

ID_VENTA	PRODUCTO	CANTIDAD	PRECIOUNITARIO	SUBTOTAL
241	Zapatilla Running Air Max	2	89.99	179.98
1	Sandalia Deportiva Trail	6	38	228
1	Zapato Formal Oxford Cuero	16	75	1200
1	Bota Alta Cuero Invierno	20	55	1100
1	Bota de Trabajo Resistente	19	60	1140
2	Zapatilla Running Air Max	6	89.99	539.94
3	Zapato Formal Derby Negro	1	80	80
4	Sandalia Deportiva Trail	4	38	152
4	Zapatilla Infantil Colors	2	28.5	57

4.9. Productos con proveedor (JOIN)

Combina producto y proveedor para mostrar de qué proveedor y país proviene cada producto.

```
SELECT p.nombre AS producto,  
       pr.tipo_proveedor AS tipo_rpdcto,  
       pr.nombre AS proveedor,  
       pr.pais AS pais_proveedor  
FROM producto p  
JOIN proveedor pr ON p.id_proveedor = pr.id_proveedor  
ORDER BY p.nombre;
```

Se han recuperado todas las filas: 25 en 0.010 segundos

	PRODUCTO	TIPO_RPDUCTO	PROVEEDOR	PAIS_PROVEEDOR
1	Bota Alta Cuero Invierno	Nacional	Cuero y Calzado Latacunga	Ecuador
2	Bota de Lluvia Impermeable	Nacional	Calzado Cuenca Export	Ecuador
3	Bota de Trabajo Resistente	Nacional	Fabrica de Calzado Tungurahua	Ecuador
4	Bota Outdoor Impermeable X	Importado	Puma South America	Brasil
5	Bota Outdoor Montana	Importado	Guangzhou Footwear Export	China
6	Bota Outdoor Trekking Pro	Importado	Grupo Calzado del Pacifico	Colombia
7	Sandalia Casual Verano	Nacional	Buestan Calzado	Ecuador
8	Sandalia Deportiva Trail	Importado	New Balance Import Co	Estados Unidos
9	Sandalia Playera Comfort	Nacional	Venus Calzado Cia Ltda	Ecuador

4.10. Devoluciones con cliente y producto (JOIN)

Combina devolucion, cliente y producto para mostrar qué cliente devolvió qué producto, cuándo y por qué motivo.

```
SELECT cl.nombre || ' ' || cl.apellido AS cliente,  
       p.nombre AS producto,  
       d.fecha_devolucion AS fecha_devolucion,  
       d.motivo,  
       d.cantidad  
FROM devolucion d  
JOIN venta v ON d.id_venta = v.id_venta  
JOIN cliente cl ON v.id_cliente = cl.id_cliente  
JOIN producto p ON d.id_producto = p.id_producto  
ORDER BY d.fecha_devolucion DESC;
```

Se han recuperado todas las filas: 25 en 0.020 segundos

	CLIENTE	PRODUCTO	FECHA_DEVOLUCION	MOTIVO	CANTIDAD
1	Danilo Guaman	Bota Alta Cuero Invierno	21/07/26	Talla incorrecta	2
2	Patricio Pena	Zapato Formal Derby Negro	18/07/26	Empaque danado	2
3	Emilio Castillo	Bota Outdoor Trekking Pro	17/07/26	Producto equivocado entregado	3
4	Gabriela Leon	Sandalia Deportiva Trail	15/07/26	Producto en mal estado	3
5	Ruben Cadena	Zapato Formal Loafer	10/07/26	Producto en mal estado	1
6	Fernanda Ponce	Zapato Formal Loafer	05/07/26	Producto en mal estado	1
7	Monica Rosero	Zapatilla Urbana Chunky	26/06/26	Producto equivocado entregado	2
8	Gabriela Leon	Bota Outdoor Montana	25/06/26	Cliente cambio de opinion	3
9	Bolivar Yanez	Zapatilla Infantil Colors	17/06/26	Producto equivocado entregado	1

4.11. Total vendido por vendedor (GROUP BY)

Agrupar las ventas por vendedor (empleado) para calcular cuántas ventas y cuánto dinero generó cada uno.

```
SELECT
    e.nombre || ' ' || e.apellido AS vendedor,
    COUNT(v.id_venta) AS cantidad_ventas,
    SUM(v.total) AS total_vendido
FROM empleado e
LEFT JOIN venta v ON e.id_empleado = v.id_empleado
WHERE e.cargo = 'Vendedor'
GROUP BY
    e.id_empleado,
    e.nombre,
    e.apellido
ORDER BY total_vendido DESC;
```

Se han recuperado todas las filas: 3 en 0.040 segundos

	VENDEDOR	CANTIDAD_VENTAS	TOTAL_VENDIDO
1	Paola Vega	5	4015.46
2	Lucia Molina	3	2840.45
3	Luis Vasquez	2	1351.22

4.12. Productos más vendidos (GROUP BY)

Agrupar detalle_venta por producto para identificar los productos con mayor cantidad de unidades vendidas.

```
SELECT p.nombre AS producto,
    SUM(dv.cantidad) AS total_unidades_vendidas
FROM detalle_venta dv
JOIN producto p ON dv.id_producto = p.id_producto
GROUP BY p.nombre
ORDER BY TotalUnidadesVendidas DESC;
```

Se han recuperado todas las filas: 25 en 0.016 segundos

	PRODUCTO	TOTAL_UNIDADES_VENDIDAS
1	Sandalia Casual Verano	116
2	Bota Alta Cuero Invierno	94
3	Bota de Lluvia Impermeable	67
4	Zapato Formal Derby Negro	56
5	Sandalia Deportiva Trail	53
6	Zapato Casual Mocasín	47
7	Bota Outdoor Montana	46
8	Bota Outdoor Trekking Pro	45
9	Zapatilla Running Lite	39

4.13. Ventas por mes (GROUP BY)

Agrupar las ventas por mes para mostrar la evolución del total vendido a lo largo del tiempo.

```
SELECT
    EXTRACT( MONTH FROM fecha_venta) AS mes,
    COUNT(*) AS ventas_realizadas,
    SUM(total) AS total_ingreso
FROM venta
GROUP BY mes
ORDER BY mes DESC;
```

Se han recuperado todas las filas: 7 en 0.027 segundos

	MES	VENTAS_REALIZADAS	TOTAL_INGRESO
1	7	10	15944.21
2	6	7	10055.79
3	5	10	14772.29
4	4	4	1958.85
5	3	8	8437.83
6	2	10	11991.94
7	1	6	3831.42

4.14. Compras por cliente (GROUP BY)

Agrupar las ventas por cliente para mostrar cuántas compras y qué monto acumulado tiene cada uno.

```
SELECT
    c.nombre || ' ' || c.apellido AS cliente,
    COUNT(v.id_venta) AS cantidad_compras,
    NVL(SUM(v.total),0) AS monto_acumulado
FROM cliente c
LEFT JOIN venta v ON c.id_cliente = v.id_cliente
GROUP BY
    c.id_cliente,
    c.nombre,
    c.apellido
ORDER BY monto_acumulado DESC;
```

Se han recuperado todas las filas: 37 en 0.021 segundos

	CLIENTE	CANTIDAD_COMPRAS	MONTO_ACUMULADO
1	Gabriela Leon	3	8403.18
2	Silvia Loja	4	7522.23
3	Monica Rosero	3	7383.16
4	Esteban Naranjo	2	4773.07
5	Karla Mendoza	1	4364.25
6	Ivan Cardenas	1	4019.62
7	Cristina Paredes	5	3771.67
8	Daniela Morales	4	3689.5
9	Mateo Herrera	2	3117.07

4.15. Devoluciones por vendedor (GROUP BY)

Agrupar las devoluciones por vendedor para identificar quién tiene más devoluciones asociadas a sus ventas.

```
SELECT
    e.nombre || ' ' || e.apellido AS vendedor,
    COUNT(d.id_devolucion) AS total_devoluciones
FROM devolucion d
JOIN venta v ON d.id_venta = v.id_venta
JOIN empleado e ON v.id_empleado = e.id_empleado
WHERE e.cargo = 'Vendedor'
GROUP BY e.nombre, e.apellido
ORDER BY total_devoluciones DESC;
```

	VENDEDOR	TOTAL_DEVOLUCIONES
1	Paola Vega	3
2	Lucia Molina	2

4.16. Clientes con compras superiores al promedio (Subconsulta)

Usa una subconsulta con el promedio general de compras para filtrar solo los clientes que gastan por encima de ese promedio.

```
SELECT
    CONCAT(c.nombre, ' ', c.apellido) AS nombre_cliente,
    SUM(v.total) AS total_comprado
FROM cliente c
JOIN venta v ON v.id_cliente = c.id_cliente
GROUP BY c.id_cliente, c.nombre, c.apellido
HAVING SUM(v.total) > (
    SELECT
        AVG(total)
    FROM venta
);
```

Se han recuperado todas las filas: 18 en 0.029 segundos

	NOMBRE_CLIENTE	TOTAL_COMPRADO
1	Gabriela Leon	8403.18
2	Daniela Morales	3689.5
3	Cristina Paredes	3771.67
4	Tatiana Merino	2052.15
5	Fernanda Ponce	2576.11
6	Silvia Loja	7522.23
7	Bolivar Yanez	1379.78
8	Roberto Castro	1938.17
9	Ruben Cadena	2215.05

4.17. Productos con precio mayor al promedio (Subconsulta)

Usa una subconsulta con el precio promedio de los productos para listar solo los que cuestan más que ese promedio.

```
SELECT nombre AS producto,  
       precio AS precio  
FROM producto  
WHERE precio > (SELECT AVG(precio) FROM producto)  
ORDER BY precio DESC;
```

Se han recuperado todas las filas: 12 en 0.025 segundos

	PRODUCTO	PRECIO
1	Bota Outdoor Impermeable X	120
2	Bota Outdoor Montana	115
3	Bota Outdoor Trekking Pro	110
4	Zapatilla Running Elite	98
5	Zapatilla Running Boost	95
6	Zapatilla Running Air Max	89.99
7	Zapato Formal Derby Negro	80
8	Zapatilla Running Lite	79.99
9	Zapato Formal Oxford Cuero	75

4.18. Vendedores con ventas superiores al promedio (Subconsulta)

Usa una subconsulta con el promedio de ventas por vendedor para mostrar solo a los que superan ese promedio.

```
SELECT  
       e.nombre||' '||e.apellido AS vendedor,  
       SUM(v.total) AS total_vendido  
FROM empleado e  
JOIN venta v ON e.id_empleado = v.id_empleado  
WHERE e.cargo = 'Vendedor'  
GROUP BY e.id_empleado, e.nombre, e.apellido  
HAVING SUM(v.total) > (  
    SELECT AVG(total_vendido)  
    FROM (  
        SELECT  
            v2.id_empleado,  
            SUM(v2.total) AS total_vendido  
        FROM venta v2  
        JOIN empleado e2 ON v2.id_empleado = e2.id_empleado  
        WHERE e2.cargo = 'Vendedor'  
        GROUP BY v2.id_empleado))  
ORDER BY total_vendido DESC;
```

Se han recuperado todas las filas: 2 en 0.042 segundos

	VENDEDOR	TOTAL_VENDIDO
1	Paola Vega	4015.46
2	Lucia Molina	2840.45

4.19. Productos que nunca se han vendido (Subconsulta)

Usa *NOT IN* en *detalle_venta* para identificar productos que nunca aparecen en ninguna venta.

```
SELECT id_producto,  
       nombre AS nombre_producto,  
       stock_actual AS sotck_actual  
FROM producto  
WHERE id_producto NOT IN (  
       SELECT DISTINCT id_producto FROM detalle_venta  
)  
ORDER BY nombre;
```

Se han recuperado todas las filas: 0 en 0.014 segundos

	ID_PRODUCTO	NOMBRE_PRODUCTO	SOTCK_ACTUAL
--	-------------	-----------------	--------------

4.20. Clientes que no han realizado compras (Subconsulta)

Usa *NOT IN* en *venta* para identificar clientes registrados que nunca han comprado.

```
SELECT  
       c.id_cliente,  
       c.nombre AS nombres,  
       c.apellido AS apellidos,  
       c.telefono  
FROM cliente c  
WHERE c.id_cliente NOT IN (  
       SELECT v.id_cliente  
       FROM venta v  
);|
```

Se han recuperado todas las filas: 12 en 0.011 segundos

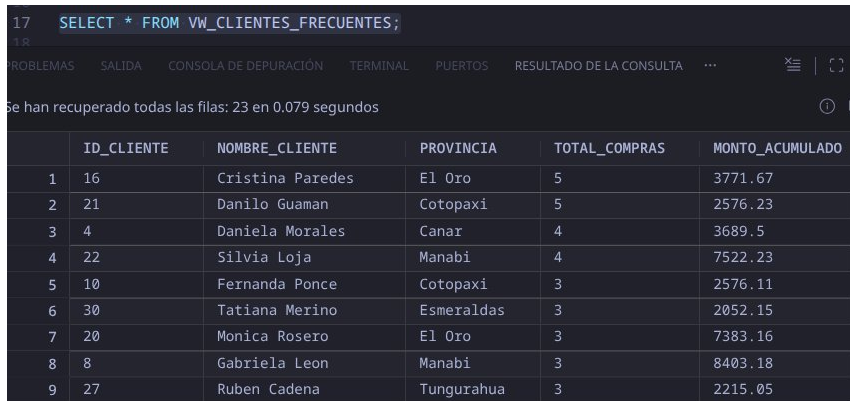
	ID_CLIENTE	NOMBRES	APELLIDOS	TELEFONO
1	23	Marco	Yopez	0919736572
2	14	Veronica	Villacis	0920117988
3	7	Sebastian	Rivera	0926240908
4	31	Gustavo	Quinde	0983227889
5	11	Ricardo	Guerrero	0998447167
6	17	Alberto	Cabrera	0984593961
7	33	Wilson	Tapia	0985146293
8	26	Johanna	Bravo	0950181935
9	28	Belen	Barros	0945431319

5. VISTAS

5.1. Clientes Frecuentes

Esta vista proporciona información de los clientes que han realizado más compras.

```
CREATE OR REPLACE VIEW vw_clientes_frecuentes
AS
SELECT
    c.id_cliente,
    CONCAT(c.nombre, ' ', c.apellido) AS nombre_cliente,
    c.provincia,
    COUNT(v.id_cliente) AS total_compras,
    NVL(SUM(v.total), 0) AS monto_acumulado
FROM cliente c
JOIN venta v ON v.id_cliente = c.id_cliente
GROUP BY c.id_cliente, c.nombre, c.apellido, c.provincia
ORDER BY total_compras DESC;
```



	ID_CLIENTE	NOMBRE_CLIENTE	PROVINCIA	TOTAL_COMPRAS	MONTO_ACUMULADO
1	16	Cristina Paredes	El Oro	5	3771.67
2	21	Danilo Guaman	Cotopaxi	5	2576.23
3	4	Daniela Morales	Canar	4	3689.5
4	22	Silvia Loja	Manabi	4	7522.23
5	10	Fernanda Ponce	Cotopaxi	3	2576.11
6	30	Tatiana Merino	Esmeraldas	3	2052.15
7	20	Monica Rosero	El Oro	3	7383.16
8	8	Gabriela Leon	Manabi	3	8403.18
9	27	Ruben Cadena	Tungurahua	3	2215.05

5.2. Ventas Consolidadas

Devuelve el resumen completo listo para mostrar: fecha, quién compró, quién vendió, tipo de venta, y los tres montos (subtotal antes de descuento, el descuento aplicado, y el total final), devuelve una fila por cliente.

```
CREATE OR REPLACE VIEW vw_ventas_consolidadas
AS
SELECT
    v.id_venta,
    v.fecha_venta,
    CONCAT(c.nombre, ' ', c.apellido) AS nombre_cliente,
    CONCAT(e.nombre, ' ', e.apellido) AS nombre_vendedor,
    v.tipo_venta,
    SUM(dv.subtotal) AS subtotal,
    v.iva,
    v.descuento,
    v.total
FROM venta v
JOIN empleado e ON e.id_empleado = v.id_empleado
JOIN cliente c ON c.id_cliente = v.id_cliente
JOIN detalle_venta dv ON dv.id_venta = v.id_venta
GROUP BY v.id_venta, c.nombre, c.apellido, e.nombre, e.apellido,
v.fecha_venta, v.tipo_venta, v.descuento, v.total, v.iva
ORDER BY v.fecha_venta DESC;
```

41 `SELECT * FROM vw_ventas_consolidadas;`

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS RESULTADOS DE LA CONSULTA RESULTADO DE LA CONSULTA SALIDA DE SCRIPT HISTORIAL SQL SUPERVISIÓN DE TAREAS

Se han recuperado todas las filas: 55 en 0.026 segundos

	ID_VENTA	FECHA_VENTA	NOMBRE_CLIENTE	NOMBRE_VENDEDOR	TIPO_VENTA	SUBTOTAL	IVA	DESCUENTO	TOTAL
1	49	25/07/26	Esteban Naranjo	Juan Rodriguez	Mayor	2425.5	363.82	0	2789.32
2	38	23/07/26	Elena Ramirez	Camila Andrade	Menor	119.7	17.95	0	137.65
3	12	21/07/26	Ivan Cardenas	Valentina Freire	Mayor	3578.54	536.78	95.7	4019.62
4	7	18/07/26	Bolivar Yanez	Lucia Molina	Menor	816	122.4	7.23	931.17
5	33	18/07/26	Gabriela Leon	Paola Vega	Mayor	1584.5	237.67	0	1822.17
6	26	15/07/26	Esteban Naranjo	Maria Gomez	Mayor	1725	258.75	0	1983.75
7	36	14/07/26	Gabriela Leon	Pedro Suarez	Mayor	2075	311.25	23.44	2362.81
8	43	05/07/26	Daniela Morales	Carlos Perez	Menor	767.96	115.19	4.32	878.83
9	37	04/07/26	Fernanda Ponce	Andres Zambrano	Menor	396.5	59.47	0	455.97

5.3. Productos con Bajo Stock

Compara stock_actual contra stock_minimo por producto para identificar cuáles necesitan reposición.

```
CREATE OR REPLACE VIEW vw_productos_bajo_stock AS
SELECT
    p.id_producto,
    p.nombre AS nombre_producto,
    c.nombre AS categoria,
    p.stock_actual,
    p.stock_minimo
FROM producto p
JOIN categoria c ON p.id_categoria = c.id_categoria
WHERE p.stock_actual < p.stock_minimo;

SELECT * FROM vw_productos_bajo_stock;
```

PROBLEMAS 1 SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS RESULTADO DE LA CONSULTA ...

Se han recuperado todas las filas: 0 en 0.014 segundos

ID_PRODUCTO	NOMBREPRODUCTO	CATEGORIA	STOCKACTUAL	STOCKMINIMO
-------------	----------------	-----------	-------------	-------------

5.4. Promociones Aplicadas

Muestra qué ventas usaron promoción, cuál fue el descuento aplicado y en qué fecha, para medir el impacto de cada promoción.

```
CREATE OR REPLACE VIEW vw_promociones_aplicadas AS
SELECT v.id_venta,
    c.nombre|| ' ' ||c.apellido AS cliente,
    p.nombre AS Promocion,
    v.descuento AS Descuento_aplicado,
    v.fecha_venta
FROM venta v
JOIN cliente c ON v.id_cliente = c.id_cliente
JOIN promocion p ON v.id_promocion = p.id_promocion;
```

303 `SELECT * FROM vw_promociones_aplicadas;`

PROBLEMAS 1 SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS RESULTADOS DE LA CONSULTA RESULTADO DE LA CONSULTA SALIDA E

Se han recuperado todas las filas: 25 en 0.020 segundos

	ID_VENTA	CLIENTE	PROMOCION	DESCUENTO_APLICADO	FECHA_VENTA
1	46	Elena Ramirez	Carnaval SuperShoes	4.45	27/04/26
2	45	Roberto Castro	Rebajas de Enero	7.63	28/01/26
3	2	Daniela Morales	Descuento Fin de Semana	10.34	15/03/26
4	41	Daniela Morales	Día de la Madre	25.74	14/06/26
5	43	Daniela Morales	Black Friday SuperShoes	4.32	05/07/26
6	35	Isabella Vera	Fiestas de Quito	8.04	26/03/26
7	36	Gabriela Leon	San Valentin	23.44	14/07/26
8	18	Emilio Castillo	San Valentin	18.31	02/06/26
9	5	Fernanda Ponce	Carnaval SuperShoes	10.81	24/05/26

5.5. Devoluciones

Presenta cada devolución junto con el cliente y producto asociado, la cantidad devuelta y el motivo.

```
CREATE OR REPLACE VIEW vw_devoluciones_aplicadas AS
SELECT d.id_devolucion,
       d.fecha_devolucion           AS Fecha,
       cl.nombre || ' ' || cl.apellido AS Cliente,
       p.nombre                     AS Producto,
       d.cantidad,
       d.motivo
FROM devolucion d
JOIN venta      v ON d.id_venta    = v.id_venta
JOIN cliente    cl ON v.id_cliente = cl.id_cliente
JOIN producto   p ON d.id_producto = p.id_producto;
```

Se han recuperado todas las filas: 25 en 0.045 segundos

	ID_DEVOLUCION	FECHA	CLIENTE	PRODUCTO	CANTIDAD	MOTIVO
1	15	08/03/26	Roberto Castro	Zapatilla Casual Urban Fit	1	Producto en mal estado
2	13	13/06/26	Roberto Castro	Bota de Lluvia Impermeable	1	Cliente cambio de opinion
3	25	25/01/26	Daniela Morales	Sandalia Casual Verano	2	Talla incorrecta
4	7	20/01/26	Daniela Morales	Zapatilla Urbana Chunky	2	Cliente cambio de opinion
5	6	05/06/26	Isabella Vera	Bota de Lluvia Impermeable	1	Cliente cambio de opinion
6	10	17/01/26	Isabella Vera	Bota Outdoor Montana	2	Producto equivocado entregado
7	21	15/07/26	Gabriela Leon	Sandalia Deportiva Trail	3	Producto en mal estado
8	2	02/01/26	Gabriela Leon	Sandalia Deportiva Trail	2	Producto en mal estado
9	17	12/01/26	Gabriela Leon	Sandalia Deportiva Trail	1	Talla incorrecta

5.6. Desempeño de Vendedores

Combina el total vendido y el total de devoluciones por vendedor para evaluar su desempeño comercial.

```
CREATE OR REPLACE VIEW vw_desempeno_vendedores AS
SELECT e.id_empleado,
       e.nombre || ' ' || e.apellido AS Empleado,
       NVL(ventas.total_ventas, 0) AS total_ventas,
       NVL(ventas.monto_generado, 0) AS monto_generado,
       NVL(devolucion.total_devoluciones, 0) AS total_devoluciones
FROM empleado e
LEFT JOIN (SELECT id_empleado,
                  COUNT(id_venta) AS total_ventas,
                  SUM(total) AS monto_generado
            FROM venta
            GROUP BY id_empleado) ventas ON e.id_empleado=ventas.id_empleado
LEFT JOIN (SELECT v.id_empleado,
                  COUNT(d.id_devolucion) AS total_devoluciones
            FROM devolucion d
            JOIN venta v ON d.id_venta = v.id_venta
            GROUP BY v.id_empleado) devolucion ON e.id_empleado=devolucion.id_e
WHERE e.cargo='Vendedor';
```

110 SELECT * FROM vw_desempeno_vendedores;

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS RESULTADO DE LA CONSULTA ...

Se han recuperado todas las filas: 3 en 0.060 segundos

	ID_EMPLEADO	EMPLEADO	TOTAL_VENTAS	MONTO_GENERADO	TOTAL_DEVOLUCIONES
1	8	Lucia Molina	3	2840.45	2
2	14	Paola Vega	5	4015.46	3
3	5	Luis Vasquez	2	1351.22	0

6 FUNCIONES

6.1 Calcular IVA

Recibe un monto (subtotal - descuento) y retorna el valor del IVA (15%) sobre ese monto.

```
CREATE OR REPLACE FUNCTION fn_calcular_iva(
    monto NUMBER
)
RETURN NUMBER
IS
iva_calculado NUMBER;
BEGIN
    IF monto IS NULL OR monto < 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'El monto no puede ser nulo o negativo');
    END IF;

    iva_calculado := ROUND(monto * 0.15, 2);
    RETURN iva_calculado;
END fn_calcular_iva;
```

```

--* Éxito
SELECT fn_calcular_iva(100) AS iva FROM dual;      -- Resultado: 15
--* Fallo
SELECT fn_calcular_iva(-50) AS iva FROM dual;      -- Resultado:
ORA-20001 "El monto no puede ser nulo o negativo"

```

IVAs	SALIDA	CONSOLA DE DEPURACIÓN	TERMINAL	PUERTOS	RESULTADO DE LA CONSULTA
1	15				

Se recuperaron todas las filas: 1 en 0.023 segundos

```

21 --* Fallo
22 SELECT fn_calcular_iva(-50) AS iva FROM dual;
23 ORA-20001 "El monto no puede ser nulo o negativo"

```

Error at Line: 1, Column: 8

SQL Error: ORA-20001: El monto no puede ser nulo o negativo
ORA-06512: en "PROYECTOBD.FN_CALCULAR_IVA", línea 9

6.2 Calcular Descuento

Recibe el subtotal y el nombre de la promoción, y retorna el descuento a aplicar según sea de tipo Porcentaje o Monto fijo.

```

CREATE OR REPLACE FUNCTION fn_calcular_descuento(
    valor NUMBER,
    nombre_promocion VARCHAR2 DEFAULT NULL)
RETURN number
IS
    v_tipo VARCHAR2(20);
    v_valor NUMBER;
    v_count NUMBER;
    v_minimo NUMBER;
    descuento NUMBER;
BEGIN
    IF valor IS NULL OR valor < 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'El valor no puede ser nulo o negativo');
    END IF;
    IF nombre_promocion IS NULL THEN
        RETURN 0;
    END IF;

    SELECT COUNT(*)
    INTO v_count
    FROM PROMOCION
    WHERE nombre = nombre_promocion;

    IF v_count = 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'La promoción no existe');
    END IF;

    SELECT tipo_descuento, valor_descuento, monto_minimo
    INTO v_tipo, v_valor, v_minimo
    FROM promocion WHERE nombre = nombre_promocion;

    IF valor < v_minimo THEN
        RETURN 0;
    END IF;
    IF v_tipo = 'Porcentaje' then
        descuento := valor*(v_valor/100);
    ELSE
        descuento := v_valor;
    END IF;

    RETURN ROUND(descuento,2);
END fn calcular descuento;

```

```

69  --* Éxito (promo 'Dia del Padre': Monto 7.23, mínimo 0, siempre aplica)
70  SELECT fn_calcular_descuento(80,'Dia del Padre') AS descuento FROM dual;

```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS RESULTADO DE LA CONSULTA ...

Se han recuperado todas las filas: 1 en 0.085 segundos

	DESCUENTO
1	7.23

```

71  --* Fallo (promoción inexistente)
72  SELECT fn_calcular_descuento(80,'Promo Inventada') AS descuento FROM dual;
73

```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS RESULTADO DE LA CONSULTA ...

Error at Line: 1, Column: 8

SQL Error: ORA-20001: La promoción no existe
ORA-06512: en "PROYECTOBD.FN_CALCULAR_DESCUENTO", línea 27
ORA-06512: en línea 1

6.3 Calcular Antigüedad del Cliente

Recibe la fecha de registro de un cliente y retorna su antigüedad (años) desde esa fecha hasta la fecha actual.

```

CREATE OR REPLACE FUNCTION fn_calcular_antigüedad_cliente(
    C_id_cliente NUMBER
)
RETURN VARCHAR2
IS
    resultado VARCHAR2(20);
BEGIN
    SELECT
        TRUNC (
            MONTHS_BETWEEN(SYSDATE, fecha_registro)/12
        ) || ' ' || 'Años'
    INTO resultado
    FROM cliente
    WHERE id_cliente = C_id_cliente;

    RETURN resultado;
END;

```

```

83
84  --* Éxito
85  SELECT fn_calcular_antigüedad_cliente(1) AS antigüedad FROM dual;

```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS RESULTADO DE LA CONSULTA ...

Se han recuperado todas las filas: 1 en 0.011 segundos

	ANTIGÜEDAD
1	2 Años

```
--* Fallo (cliente que no existe)
SELECT fn_calcular_antiguedad_cliente(9999) AS antiguedad FROM dual;
```

han recuperado todas las filas: 1 en 0.024 segundos

	ANTIGUEDAD
1	(null)

6.4 Calcular Comisión

Recibe el total vendido por un empleado y retorna el valor de su comisión según un porcentaje definido.

```
CREATE OR REPLACE FUNCTION fn_calcular_comision(
    p_id_empleado IN NUMBER,
    p_porcentaje IN NUMBER DEFAULT 5
) RETURN NUMBER IS
    v_total NUMBER;
BEGIN
    SELECT NVL(SUM(total), 0)
    INTO v_total
    FROM venta
    WHERE id_empleado = p_id_empleado;
    RETURN ROUND(v_total * p_porcentaje / 100, 2);
EXCEPTION
    WHEN OTHERS THEN RETURN 0;
END fn_calcular_comision;
/

SELECT fn_calcular_comision(1, 8) AS Comision;
```

han recuperado todas las filas: 1 en 0.021 segundos

	COMISION
1	419.45

```
SELECT fn_calcular_comision(9999, 8) AS comision FROM dual;
```

han recuperado todas las filas: 1 en 0.023 segundos

	COMISION
1	0

6.5 Productos Bajo Stock

Retorna la cantidad de productos cuyo stock_actual es menor a su stock_minimo.

```
CREATE OR REPLACE FUNCTION fn_productos_bajo_stock
RETURN NUMBER IS
    v_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_count
    FROM producto
    WHERE stock_actual < stock_minimo;
    RETURN v_count;
END fn_productos_bajo_stock;
/
```

--* Exito

```
SELECT fn_productos_bajo_stock AS productos_bajo_stock FROM dual;
```

Se han recuperado todas las filas: 1 en 0.009 segundos

	PRODUCTOS_BAJO_STOCK
1	0

6.6 Clientes Frecuentes

Retorna si un cliente es 'frecuente' según la regla de negocio (más de 5 compras o monto acumulado $\geq$ \$500).

```
CREATE OR REPLACE FUNCTION Clientes_Frecuentes(
    f_cedula_cliente NUMBER
)
RETURN VARCHAR2
IS
    numero_compras NUMBER;
    monto_acumulado DECIMAL(10,2);
BEGIN
    SELECT COUNT(id_venta),
           NVL(SUM(total),0)
    INTO numero_compras,
        monto_acumulado
    FROM venta v
    JOIN cliente c ON c.id_cliente=v.id_cliente
    WHERE c.cedula=f_cedula_cliente;

    IF numero_compras>5 OR monto_acumulado>=500 THEN
        RETURN 'Cliente frecuente';
    ELSE
        RETURN 'Cliente no frecuente';
    END IF;
END;
```

```
SELECT Clientes_Frecuentes('1702693824') AS Estado;
```

Se han recuperado todas las filas: 1 en 0.032 segundos

	ESTADO
1	Cliente no frecuente

```
--* Éxito
SELECT Clientes_Frecuentes('17936043811') AS estado FROM dual;
```

han recuperado todas las filas: 1 en 0.021 segundos	
	ESTADO
1	Cliente frecuente

7. PROCEDIMIENTOS ALMACENADOS

7.1. Registrar Cliente

Valida que ningún campo obligatorio venga vacío, que el correo tenga '@', y que la cédula y el correo no estén ya registrados; si todo es correcto, inserta el nuevo cliente.

```
CREATE OR REPLACE PROCEDURE sp_registrar_cliente(
    c_cedula VARCHAR2,
    c_nombre VARCHAR2,
    c_apellido VARCHAR2,
    c_telefono VARCHAR2,
    c_correo VARCHAR2,
    c_provincia VARCHAR2,
    c_direccion VARCHAR2
)
IS
    c_cedula_existe NUMBER;
    c_correo_existe NUMBER;
BEGIN
    IF c_cedula IS NULL THEN
        RAISE_APPLICATION_ERROR(-20001,'La cedula no puede estar vacía');

    ELSIF c_nombre IS NULL THEN
        RAISE_APPLICATION_ERROR(-20001,'El nombre no puede estar vacío');

    ELSIF c_apellido IS NULL THEN
        RAISE_APPLICATION_ERROR(-20001,'El apellido no puede estar vacío');

    ELSIF c_telefono IS NULL THEN
        RAISE_APPLICATION_ERROR(-20001,'EL teléfono no puede estar vacío');

    ELSIF c_correo IS NULL THEN
        RAISE_APPLICATION_ERROR(-20001,'El correo no puede estar vacío');

    ELSIF c_provincia IS NULL THEN
        RAISE_APPLICATION_ERROR(-20001,'La provincia no puede estar vacía');

    ELSIF c_direccion IS NULL THEN
        RAISE_APPLICATION_ERROR(-20001,'La dirección no puede estar vacía');
    END IF;

    IF INSTR(c_correo, '@') = 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'El correo debe contener @');
    END IF;
```

```

SELECT
COUNT(cedula)
INTO c_cedula_existe
FROM cliente
WHERE cedula = c_cedula;

SELECT
COUNT(correo)
INTO c_correo_existe
FROM cliente
WHERE correo = c_correo;

IF c_cedula_existe > 0 THEN
    RAISE_APPLICATION_ERROR(-20001, 'La cédula ya se encuentra registrada,
    intente nuevamente');
ELSIF c_correo_existe > 0 THEN
    RAISE_APPLICATION_ERROR(-20001, 'El correo ya se encuentra registrado,
    intente nuevamente');
END IF;

```

```

INSERT INTO cliente(cedula,nombre,apellido,telefono,correo,provincia,
direccion,fecha_registro)
VALUES(c_cedula,c_nombre,c_apellido,c_telefono,c_correo,c_provincia,
c_direccion,SYSDATE);

COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END;

--* Éxito
EXEC sp_registrar_cliente('1798631205', 'Leonel', 'Messi', '0963258971', 'leo.
messi@correo.com', 'Quito', 'Av. América');
SELECT * FROM cliente WHERE cedula = '1798631205';

```

recuperado todas las filas: 1 en 0.011 segundos

ID_CLIENTE	CEDULA	NOMBRE	APELLIDO	TELEFONO	CORREO
222	1798631205	Leonel	Messi	0963258971	leo.messi@correo.com

```

74 --* Fallo (cédula ya registrada)
75 EXEC sp_registrar_cliente('17936043811', 'Duplicado', 'Prueba', '0999999999',
    'otro@correo.com', 'Pichincha', 'Calle Falsa 123');
76
77
Procedimiento PL/SQL terminado correctamente.
BEGIN sp_registrar_cliente('17936043811', 'Duplicado', 'Prueba', '0999999999', 'otro@correo.
ERROR en la línea 1:
ORA-20001: La cédula ya se encuentra registrada, intente nuevamente

```

7.2. Registrar Producto

Valida que no exista ya un producto con el mismo nombre, inserta el producto y registra automáticamente su stock inicial como un movimiento de tipo 'Entrada' en inventario..

```
CREATE OR REPLACE PROCEDURE sp_registrar_producto(
    p_nombre IN VARCHAR2,
    p_id_categoria IN NUMBER,
    p_id_proveedor IN NUMBER,
    p_precio IN NUMBER,
    p_stock IN NUMBER,
    p_stock_min IN NUMBER
) AS
    v_existe NUMBER;
    v_id NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_existe
    FROM producto
    WHERE UPPER(nombre) = UPPER(p_nombre);

    IF v_existe > 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Ya existe un producto con ese nombre.
    ');
    END IF;

    INSERT INTO producto(nombre, id_categoria, id_proveedor, precio,
        stock_actual, stock_minimo)
    VALUES (p_nombre, p_id_categoria, p_id_proveedor, p_precio, p_stock,
        p_stock_min)
    RETURNING id_producto INTO v_id;

    INSERT INTO inventario(id_producto, tipo_movimiento, cantidad, motivo)
    VALUES (v_id, 'Entrada', p_stock, 'Stock inicial al registrar producto');

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Producto registrado. ID: ' || v_id);
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END sp_registrar_producto;
/
```

```
15 --* Éxito
16 EXEC sp_registrar_producto('Bota Impermeable Trekking', 1, 7, 68.50, 25, 8);
17 SELECT * FROM producto WHERE nombre = 'Bota Impermeable Trekking';
18 SELECT * FROM inventario WHERE motivo = 'Stock inicial al registrar producto';
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS RESULTADO DE LA CONSULTA ...

Se han recuperado todas las filas: 1 en 0.020 segundos

	ID_PRODUCTO	NOMBRE	ID_CATEGORIA	ID_PROVEEDOR	PRECIO
1	241	Bota Impermeable Trekking	1	7	68.5

```

115 --* Éxito
116 EXEC sp_registrar_producto('Bota Impermeable Trekking', 1, 7, 68.50, 25, 8);
117 SELECT * FROM producto WHERE nombre = 'Bota Impermeable Trekking';
118 SELECT * FROM inventario WHERE motivo = 'Stock inicial al registrar producto'
    ORDER BY id movimiento DESC;

```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS RESULTADO DE LA CONSULTA ...

Se han recuperado todas las filas: 2 en 0.014 segundos

	TIPO_MOVIMIENTO	CANTIDAD	FECHA_MOVIMIENTO	MOTIVO
1	Entrada	25	22/07/26	Stock inicial al registrar producto

7.3. Registrar Venta

Recibe cliente, empleado, tipo de venta y una lista de productos; valida stock disponible, inserta la cabecera en venta y el detalle en detalle_venta (el descuento de stock lo realiza el trigger trg_descontar_stock), aplica la promoción indicada y calcula el IVA y el total final.

```

CREATE OR REPLACE TYPE t_producto_venta AS OBJECT (
    nombre_producto VARCHAR2(100),
    cantidad NUMBER
);

CREATE OR REPLACE TYPE t_lista_productos AS TABLE OF t_producto_venta;

CREATE OR REPLACE PROCEDURE sp_registrar_venta(
    p_cedula_cliente IN VARCHAR2,
    p_id_empleado IN NUMBER,
    p_tipo_venta IN VARCHAR2,
    p_nombre_promocion IN VARCHAR2 DEFAULT NULL,
    p_productos IN t_lista_productos,
    p_id_venta OUT NUMBER
)
IS
    v_id_cliente cliente.id_cliente%TYPE;
    v_id_prod producto.id_producto%TYPE;
    v_precio producto.precio%TYPE;
    v_stock producto.stock_actual%TYPE;
    v_subtotal NUMBER := 0;
    v_descuento NUMBER := 0;
    v_iva NUMBER := 0;
    v_count_emp NUMBER;

BEGIN
    IF p_productos IS NULL OR p_productos.COUNT = 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'La venta debe incluir al menos un producto.');
```

```

FOR i IN 1..p_productos.COUNT LOOP

    IF p_productos(i).cantidad <= 0 THEN
        RAISE_APPLICATION_ERROR(-20001,
            'Cantidad inválida para "' || p_productos(i).
            nombre_producto || '"');
    END IF;

    SELECT id_producto, precio, stock_actual
    INTO v_id_prod, v_precio, v_stock
    FROM producto
    WHERE nombre = p_productos(i).nombre_producto;

    IF v_stock < p_productos(i).cantidad THEN
        RAISE_APPLICATION_ERROR(-20001,
            'Stock insuficiente para "' || p_productos(i).
            nombre_producto || '"');
    END IF;

    INSERT INTO detalle_venta (id_venta, id_producto, cantidad,
        precio_unitario, subtotal)
    VALUES (p_id_venta, v_id_prod, p_productos(i).cantidad, v_precio,
        v_precio * p_productos(i).cantidad);

    v_subtotal := v_subtotal + (v_precio * p_productos(i).cantidad);
END LOOP;

sp_aplicar_promocion(p_nombre_promocion, v_subtotal, v_descuento);

v_iva := fn_calcular_iva(v_subtotal - v_descuento);

UPDATE venta
SET descuento = v_descuento,
    iva = v_iva,
    total = v_subtotal - v_descuento + v_iva,
    id_promocion = (SELECT id_promocion FROM promocion WHERE nombre =
        p_nombre_promocion)
WHERE id_venta = p_id_venta;

COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END sp_registrar_venta;
/

```

```

254 DECLARE
255     v_productos t_lista_productos := t_lista_productos(
256         t_producto_venta('Zapatilla Running Air Max', 2)
257     );
258     v_id_venta NUMBER;
259 BEGIN
260     sp_registrar_venta(
261         p_cedula_cliente => '17936043811',
262         p_id_empleado     => 5,
263         p_tipo_venta       => 'Menor',
264         p_nombre_promocion => 'Dia del Padre',
265         p_productos        => v_productos,
266         p_id_venta         => v_id_venta
267     );
268 END;
269 /
270
271 $ F-11a (cantidad vendida el stock de producto)

```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS SALIDA DE S

Venta registrada con ID: 241

```

DECLARE
    v_productos t_lista_productos := t_lista_productos(
        t_producto_venta('Zapatilla Running Air Max', 999999)
    );
    v_id_venta NUMBER;
BEGIN
    sp_registrar_venta(
        p_cedula_cliente => '17936043811',
        p_id_empleado     => 5,
        p_tipo_venta      => 'Menor',
        p_nombre_promocion => NULL,
        p_productos       => v_productos,
        p_id_venta        => v_id_venta
    );
END;
/

```

EMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS SALIDA DE SCRIPT

R en la línea 1:
20001: Stock insuficiente para "Zapatilla Running Air Max".

7.4. Registrar Devolución

Valida que el producto realmente pertenezca a la venta indicada y registra la devolución (el incremento de stock lo realiza el trigger *trg_incrementar_stock*).

```

CREATE OR REPLACE PROCEDURE sp_registrar_devolucion(
    p_id_venta IN NUMBER,
    p_id_producto IN NUMBER,
    p_cantidad IN NUMBER,
    p_motivo IN VARCHAR2
) AS
    v_en_venta NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_en_venta
    FROM detalle_venta
    WHERE id_venta = p_id_venta AND id_producto = p_id_producto;

    IF v_en_venta = 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'El producto no pertenece a esa venta.
    ');
    END IF;

    INSERT INTO devolucion(id_venta, id_producto, cantidad, fecha_devolucion,
        motivo)
    VALUES (p_id_venta, p_id_producto, p_cantidad, SYSDATE, p_motivo);

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Devolucion registrada correctamente.
');
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END sp_registrar_devolucion;
/

```

```

20  --* Éxito
21  EXEC sp_registrar_devolucion(3, 8, 1, 'Cambio de color');
22  SELECT * FROM devolucion WHERE motivo = 'Cambio de color';
23

```

Problemas SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS RESULTADO DE LA CONSULTA ...

Se han recuperado todas las filas: 1 en 0.011 segundos

	ID_DEVOLUCION	ID_VENTA	ID_PRODUCTO	CANTIDAD	FECHA_DEVOLUCION	MOTIVO
1	241	3	8	1	22/07/26	Cambio de

```

325  --* Fallo (producto que no pertenece a esa venta)
326  EXEC sp_registrar_devolucion(1, 99999, 1, 'Prueba de fallo');
327

```

Problemas SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS SALIDA DE SCRIPT

Procedimiento PL/SQL terminado correctamente.
 BEGIN sp_registrar_devolucion(1, 99999, 1, 'Prueba de fallo'); END;
 *
 ERROR en la línea 1:
 ORA-20001: El producto no pertenece a esa venta.

7.5. Aplicar Promoción

Recibe el subtotal y el nombre de la promoción, y devuelve el descuento calculado apoyándose en la función `fn_calcular_descuento`.

```

CREATE OR REPLACE PROCEDURE sp_aplicar_promocion(
  p_nombre_promocion IN VARCHAR2,
  p_subtotal IN NUMBER,
  p_descuento OUT NUMBER
)
IS
BEGIN
  p_descuento := fn_calcular_descuento(p_subtotal, p_nombre_promocion);
END sp_aplicar_promocion;
/

```

```

341  --* Éxito
342  DECLARE
343    v_descuento NUMBER;
344  BEGIN
345    sp_aplicar_promocion('San Valentin', 80, v_descuento);
346    DBMS_OUTPUT.PUT_LINE('Descuento calculado: ' || v_descuento);
347  END;
348  /

```

Problemas SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS SALIDA DE SCRIPT ...

Descuento calculado: 7.65

```
350  --* Fallo (promoción inexistente)
351  DECLARE
352      v_descuento NUMBER;
353  BEGIN
354      sp_aplicar_promocion('Promo Que No Existe', 80, v_descuento);
355  END;
356  /
357
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS SALIDA DE SCRIPT ...

ORA-20001: La promoción no existe

7.6. Calcular Ventas Mensuales

Recibe mes y año, valida que sean válidos, y muestra el total vendido en ese periodo consultando la tabla venta.

```
CREATE OR REPLACE PROCEDURE sp_calcular_Ventas_Mensuales (
    v_mes NUMBER,
    v_año NUMBER
)
IS
    v_total NUMBER;
BEGIN
    IF v_mes < 1 OR v_mes > 12 THEN
        RAISE_APPLICATION_ERROR(-20001, 'El mes debe estar entre 1 y 12');
    END IF;

    IF v_año < 2000 OR v_año > EXTRACT(YEAR FROM SYSDATE) THEN
        RAISE_APPLICATION_ERROR(-20002, 'El año ingresado no es valido');
    END IF;

    SELECT NVL(SUM(total), 0)
    INTO v_total
    FROM venta
    WHERE EXTRACT(MONTH FROM fecha_venta) = v_mes
    AND EXTRACT(YEAR FROM fecha_venta) = v_año;

    IF v_total = 0 THEN
        RAISE_APPLICATION_ERROR(-20003, 'No existen ventas para el mes y año indicados');
    END IF;

    DBMS_OUTPUT.PUT_LINE(
        'Mes: ' || v_mes ||
        ' - Año: ' || v_año ||
        ' - Total ventas: $' || v_total
    );
END;
/
```

```
394 EXEC sp_calcular_ventas_mensuales(4, 2026);
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS

Procedure SP_CALCULAR_VENTAS_MENSUALES compilado

Mes: 4 - Año: 2026 - Total ventas: \$1958.85

```
397 EXEC sp_calcular_ventas_mensuales(13, 2026);
398
PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS

ORA-20001: El mes debe estar entre 1 y 12
```

7.7. Registrar Proveedor

Valida que el tipo de proveedor sea 'Nacional' o 'Importado' y que el teléfono tenga 10 dígitos, e inserta el nuevo proveedor.

```
CREATE OR REPLACE PROCEDURE sp_registrar_Proveedor (
    p_nombre IN VARCHAR2,
    p_pais IN VARCHAR2,
    p_tipo_proveedor IN VARCHAR2 ,
    p_telefono IN VARCHAR2,
    p_correo IN VARCHAR2,
    p_direccion IN VARCHAR2
)
IS
BEGIN
    IF p_tipo_proveedor NOT IN ('Nacional', 'Importado') THEN
        RAISE_APPLICATION_ERROR(-20001, 'El tipo de proveedor no es valido');
    END IF;

    IF LENGTH(p_telefono) <> 10 THEN
        RAISE_APPLICATION_ERROR(-20002, 'El teléfono debe tener 10 dígitos' );
    END IF;

    INSERT INTO proveedor(nombre,pais,tipo_proveedor,telefono,correo,
        direccion)

    VALUES (p_nombre,p_pais,p_tipo_proveedor,p_telefono,p_correo,p_direccion
    );
    COMMIT;
    EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END;
/
```

```
--* Éxito
EXEC sp_registrar_proveedor('Calzado del Pacifico', 'Ecuador', 'Nacional',
'0991112233', 'contacto@pacifico.com', 'Guayaquil, Av. 9 de Octubre');
SELECT * FROM proveedor WHERE nombre = 'Calzado del Pacifico';
```

Se recuperado todas las filas: 1 en 0.018 segundos

	ID_PROVEEDOR	NOMBRE	PAIS	TIPO_PROVEEDOR	TELEFONO
1	41	Calzado del Pacifico	Ecuador	Nacional	0991112233

```
--* Fallo (tipo de proveedor inválido)
EXEC sp_registrar_proveedor('Proveedor Prueba', 'Peru', 'Extranjero',
'0991112233', 'x@x.com', 'Direccion X');

7.8. Actualizar Inventario

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS SALIDA DE SCRIPT ...

A-20001: El tipo de proveedor no es valido
```

7.8. Actualizar Inventario

Registra manualmente un movimiento de inventario (Entrada o Salida), valida que no se genere stock negativo en una salida, y actualiza stock_actual en producto.

```
CREATE OR REPLACE PROCEDURE sp_actualizar_inventario(
    p_id_producto IN NUMBER,
    p_tipo_movimiento IN VARCHAR2,
    p_cantidad IN NUMBER,
    p_motivo IN VARCHAR2
) AS
    v_stock NUMBER;
BEGIN
    SELECT stock_actual INTO v_stock
    FROM producto WHERE id_producto = p_id_producto;

    IF p_tipo_movimiento = 'Salida' AND v_stock < p_cantidad THEN
        RAISE_APPLICATION_ERROR(-20001, 'Stock insuficiente para registrar la salida.');
```

```
EXEC sp_actualizar_inventario(1, 'Entrada', 15, 'Reposición manual de bodega');
SELECT stock_actual FROM producto WHERE id_producto = 1;

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS RESULTADO DE LA CONSULTA ...

han recuperado todas las filas: 1 en 0.026 segundos

STOCK_ACTUAL
1 59
```

```
481 EXEC sp_actualizar_inventario(1, 'Salida', 999999, 'Prueba de salida
482 excesiva');
...
PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS SALIDA DE SCRIPT ...

Procedimiento PL/SQL terminado correctamente.
Error: ORA-20001: Stock insuficiente para registrar la salida.
```

7.9. Registrar Queja

Valida que el cliente exista y que el tipo sea 'Reclamo' o 'Sugerencia', e inserta la queja/sugerencia.

```
CREATE OR REPLACE PROCEDURE sp_registrar_queja (
    q_id_cliente IN NUMBER,
    q_tipo IN VARCHAR2,
    q_descripcion IN VARCHAR2
)
IS
    id_cliente_existe NUMBER;
BEGIN
    SELECT COUNT(*)
    INTO id_cliente_existe
    FROM cliente
    WHERE id_cliente=q_id_cliente;

    IF id_cliente_existe=0 THEN
        RAISE_APPLICATION_ERROR(-20001,'El cliente ingresado no existe.');
```

```
EXEC sp_registrar_queja(1, 'Sugerencia', 'Agregar más variedad de tallas
grandes');
SELECT * FROM queja ORDER BY id_queja DESC;
...
n recuperado todas las filas: 2 en 0.020 segundos

ID_QUEJA ID_CLIENTE TIPO DESCRIPCION FECH
1 21 1 Sugerencia Agregar más variedad de tallas grandes 22/8
```

```

527 --* Fallo (cliente que no existe)
528 EXEC sp_registrar_queja(999999, 'Reclamo', 'Prueba de fallo');
529
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS  SALIDA DE SCRIPT
ERROR en la línea 1:
ORA-20001: El cliente ingresado no existe.

```

7.10. Registrar Auditoría

Procedimiento de uso interno (invocado por los triggers de auditoría) que inserta un registro en auditoria usando una transacción autónoma, para que el log no se pierda aunque la transacción principal falle.

```

CREATE OR REPLACE PROCEDURE sp_registrar_auditoria (
    a_accion IN VARCHAR2,
    a_tabla_afectada IN VARCHAR2,
    a_valor_anterior IN VARCHAR2 DEFAULT NULL,
    a_valor_nuevo IN VARCHAR2 DEFAULT NULL
)
IS
PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
    IF a_accion IS NULL OR a_accion NOT IN ('INSERT', 'UPDATE', 'DELETE') THEN
        RAISE_APPLICATION_ERROR(-20001, 'La accion debe ser INSERT, UPDATE o DELETE');
    END IF;

    IF a_tabla_afectada IS NULL THEN
        RAISE_APPLICATION_ERROR(-20002, 'La tabla afectada no puede estar vacia');
    END IF;

    INSERT INTO auditoria (usuario, accion, tabla_afectada, valor_anterior,
        valor_nuevo)
    VALUES (USER, a_accion, a_tabla_afectada, a_valor_anterior,
        a_valor_nuevo);

    COMMIT;
END sp_registrar_auditoria;
/

```

```

EXEC sp_registrar_auditoria('INSERT', 'producto', NULL, 'nombre=Bota
Impermeable Trekking, precio=68.50');
SELECT * FROM auditoria ORDER BY id_auditoria DESC;

```

Se han recuperado todas las filas: 19 en 0.049 segundos

	ID_AUDITORIA	USUARIO	ACCION	TABLA_AFECTADA	FECHA_HORA
1	129	PROYECTOBD	INSERT	producto	22/07/26 04:22:24.205

```

563 EXEC sp_registrar_auditoria('SELECT', 'producto', NULL, NULL);
564
PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS SALIDA DE SCRIPT ...

ERROR en la línea 1:
ORA-20001: La accion debe ser INSERT, UPDATE o DELETE

```

8. TRIGGERS

8.1. Auditoría INSERT Clientes

Se dispara después de insertar un cliente y registra en auditoria el id, nombre, apellido y correo del nuevo cliente.

```

CREATE OR REPLACE TRIGGER trg_auditoria_insert_cliente
AFTER INSERT ON cliente
FOR EACH ROW
BEGIN
    sp_registrar_auditoria(
        a_accion      => 'INSERT',
        a_tabla_afectada => 'cliente',
        a_valor_nuevo  => 'id_cliente=' || :NEW.id_cliente ||
                        ', nombre=' || :NEW.nombre ||
                        ', apellido=' || :NEW.apellido ||
                        ', correo=' || :NEW.correo
    );
END;
/

```

```

--* Exito
INSERT INTO cliente (cedula, nombre, apellido, telefono, correo, provincia, direccion)
VALUES ('1569325618', 'Trigger', 'Prueba', '0987654321', 'trigger.prueba2@correo.com', 'Loja', 'Calle Prueba 1');

SELECT * FROM auditoria WHERE tabla_afectada = 'cliente' AND accion = 'INSERT' ORDER BY id_auditoria DESC;

```

han recuperado todas las filas: 4 en 0.020 segundos

	ID_AUDITORIA	USUARIO	ACCION	TABLA_AFECTADA	FECHA_HORA	VALOR_ANTERIOR	VALOR_NUEVO
1	132	PROYECTOBD	INSERT	cliente	22/07/26 04:28:26.417	(null)	id_cliente=226, nombre=Trigger,

```

24 --* Fallo (el INSERT falla ANTES de llegar al trigger, por cédula duplicada
25 INSERT INTO cliente (cedula, nombre, apellido, telefono, correo, provincia,
    direccion)
26 VALUES ('1799887766', 'Duplicado', 'Prueba', '0987654321', 'otro.
    correo@correo.com', 'Loja', 'Calle Prueba 2');

```

Error que empieza en la línea: 1 del comando :

```

INSERT INTO cliente (cedula, nombre, apellido, telefono, correo, provincia, direccion)
VALUES ('1799887766', 'Duplicado', 'Prueba', '0987654321', 'otro.correo@correo.com', 'Loja', 'Calle
Informe de error -
ORA-00001: Se ha violado la restricción única (PROYECTOBD.SYS_C0010451) para la tabla PROYECTOBD.CL
ORA-03301: (Detalles de ORA-00001) ya existe la fila con los valores de columna (CEDULA:'1799887766

```

8.2. Auditoría UPDATE Clientes

Se dispara después de actualizar un cliente y compara campo por campo los valores anteriores y nuevos, registrando solo los que cambiaron.

```
CREATE OR REPLACE TRIGGER trg_auditoria_update_cliente
AFTER UPDATE ON cliente
FOR EACH ROW
DECLARE
    v_anterior VARCHAR2(500);
    v_nuevo     VARCHAR2(500);
BEGIN
    v_anterior :=
        (CASE WHEN :OLD.nombre != :NEW.nombre THEN 'nombre=' || :OLD.
nombre || ';' ELSE '' END) ||
        (CASE WHEN :OLD.apellido != :NEW.apellido THEN 'apellido=' || :OLD.
apellido || ';' ELSE '' END) ||
        (CASE WHEN :OLD.telefono != :NEW.telefono THEN 'telefono=' || :OLD.
telefono || ';' ELSE '' END) ||
        (CASE WHEN :OLD.correo != :NEW.correo THEN 'correo=' || :OLD.
correo || ';' ELSE '' END) ||
        (CASE WHEN :OLD.provincia != :NEW.provincia THEN 'provincia=' || :OLD.
provincia || ';' ELSE '' END) ||
        (CASE WHEN :OLD.direccion != :NEW.direccion THEN 'direccion=' || :OLD.
direccion || ';' ELSE '' END);
```

```
    v_nuevo :=
        (CASE WHEN :OLD.nombre != :NEW.nombre THEN 'nombre=' || :NEW.
nombre || ';' ELSE '' END) ||
        (CASE WHEN :OLD.apellido != :NEW.apellido THEN 'apellido=' || :NEW.
apellido || ';' ELSE '' END) ||
        (CASE WHEN :OLD.telefono != :NEW.telefono THEN 'telefono=' || :NEW.
telefono || ';' ELSE '' END) ||
        (CASE WHEN :OLD.correo != :NEW.correo THEN 'correo=' || :NEW.
correo || ';' ELSE '' END) ||
        (CASE WHEN :OLD.provincia != :NEW.provincia THEN 'provincia=' || :NEW.
provincia || ';' ELSE '' END) ||
        (CASE WHEN :OLD.direccion != :NEW.direccion THEN 'direccion=' || :NEW.
direccion || ';' ELSE '' END);

    IF LENGTH(v_anterior) > 0 THEN
        sp_registrar_auditoria(
            a_accion      => 'UPDATE',
            a_tabla_afectada => 'cliente',
            a_valor_anterior => v_anterior,
            a_valor_nuevo   => v_nuevo
        );
    END IF;
END trg_auditoria_update_cliente;
/
```

```
--* Éxito
UPDATE cliente SET telefono = '0911111111' WHERE cedula = '1799887766';
SELECT * FROM auditoria WHERE tabla_afectada = 'cliente' AND accion =
'UPDATE' ORDER BY id_auditoria DESC ;
```

MAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS RESULTADO DE LA CONSULTA ...

recuperado todas las filas: 1 en 0.015 segundos

TABLA_AFECTADA	FECHA_HORA	VALOR_ANTERIOR	VALOR_NUEVO
cliente	22/07/26 04:35:03.829	telefono=0987654321;	telefono=0911111111;

```

70 UPDATE cliente SET telefono = '0911111111' WHERE cedula = '1799887766';
71 SELECT COUNT(*) FROM auditoria WHERE tabla_afectada = 'cliente' AND accion =
72 'UPDATE';

```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS SALIDA DE SCRIPT ...

error que empieza en la línea: 1 del comando :
 INSERT INTO cliente (cedula, nombre, apellido, telefono, correo, provincia, direccion)
 VALUES ('1799887766', 'Duplicado', 'Prueba', '0987654321', 'otro.correo@correo.com', 'Loja', 'Calle
 informe de error -
 ORA-00001: Se ha violado la restricción única (PROYECTOBD.SYS_C0010451) para la tabla PROYECTOBD.CL
 ORA-03301: (Detalles de ORA-00001) ya existe la fila con los valores de columna (CEDULA:'1799887766

8.3. Auditoría DELETE Clientes

Se dispara antes de eliminar un cliente y guarda en auditoria todos sus datos, ya que después del DELETE ya no estarían disponibles.

```

CREATE OR REPLACE TRIGGER trg_auditoria_delete_cliente
BEFORE DELETE ON cliente
FOR EACH ROW
DECLARE
  v_anterior VARCHAR2(500);
BEGIN
  v_anterior :=
    'cedula=' || :OLD.cedula || ';' ||
    'nombre=' || :OLD.nombre || ';' ||
    'apellido=' || :OLD.apellido || ';' ||
    'telefono=' || :OLD.telefono || ';' ||
    'correo=' || :OLD.correo || ';' ||
    'provincia=' || :OLD.provincia || ';' ||
    'direccion=' || :OLD.direccion || ';';

  sp_registrar_auditoria(
    a_accion => 'DELETE',
    a_tabla_afectada => 'cliente',
    a_valor_anterior => v_anterior,
    a_valor_nuevo => NULL
  );
END trg_auditoria_delete_cliente;
/

```

```

--* Éxito(cliente sin ventas asociadas, se puede borrar)
DELETE FROM cliente WHERE cedula = '1799887766';
SELECT * FROM auditoria WHERE tabla_afectada = 'cliente' AND accion =
'DELETE' ORDER BY id_auditoria DESC;

```

MAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS RESULTADO DE LA CONSULTA ...

recuperado todas las filas: 1 en 0.026 segundos

ID_AUDITORIA	USUARIO	ACCION	TABLA_AFECTADA	FECHA_HORA
134	PROYECTOBD	DELETE	cliente	22/07/26 04:38:52.928

```
105 DELETE FROM cliente WHERE cedula = '17936043811';
```

Error que empieza en la línea: 1 del comando :
DELETE FROM cliente WHERE cedula = '17936043811'
Informe de error -
ORA-02292: restricción de integridad (PROYECTOBD.FK_VENTA_CLIENTE) violada - registro secundario

8.4. Descontar Stock por Venta

Se dispara después de insertar una línea en detalle_venta: descuenta la cantidad vendida de stock_actual en producto y registra el movimiento 'Salida' en inventario.

```
CREATE OR REPLACE TRIGGER trg_descontar_stock
AFTER INSERT ON detalle_venta
FOR EACH ROW
BEGIN
    UPDATE producto
    SET stock_actual = stock_actual - :NEW.cantidad
    WHERE id_producto = :NEW.id_producto;

    INSERT INTO inventario(id_producto, tipo_movimiento, cantidad, motivo)
    VALUES (:NEW.id_producto, 'Salida', :NEW.cantidad,
            'Venta automatica - id_venta: ' || :NEW.id_venta);
END trg_descontar_stock;
/
```

```
SELECT stock_actual FROM producto WHERE id_producto = 5;
```

Se han recuperado todas las filas: 1 en 0.027 segundos

	STOCK_ACTUAL
1	55

```
INSERT INTO detalle_venta (id_venta, id_producto, cantidad, precio_unitario, subtotal)
VALUES (1, 5, 2, 42.00, 84.00);

SELECT stock_actual FROM producto WHERE id_producto = 5;
```

Se han recuperado todas las filas: 1 en 0.018 segundos

	STOCK_ACTUAL
1	53

```
SELECT * FROM inventario WHERE motivo LIKE 'Venta automatica%' ORDER BY
id_movimiento DESC;
```

Se han recuperado todas las filas: 2 en 0.018 segundos

	TIPO_MOVIMIENTO	CANTIDAD	FECHA_MOVIMIENTO	MOTIVO
1	Salida	2	22/07/26	Venta automatica - id_venta: 1
2	Salida	2	22/07/26	Venta automatica - id_venta: 241

```
--* Fallo (cantidad que dejaria el stock en negativo)
INSERT INTO detalle_venta (id_venta, id_producto, cantidad, precio_unitario,
subtotal)
VALUES (1, 5, 999999, 42.00, 41999958.00);
```

Se han recuperado todas las filas: 1 en 0.019 segundos

Error en la línea de comandos : 1 Columna : 13
 Formato de error -
 Error SQL: ORA-20001: Stock negativo no permitido. Producto: Zapato Casual Mocasín | Antes: 53 | Después: -53
 A-06512: en "PROYECTOBD.TRG_CONTROL_STOCK_NEGATIVO", línea 3

8.5. Incrementar Stock por Devolución

Se dispara después de insertar una devolución: incrementa `stock_actual` en `producto` y registra el movimiento 'Entrada' en inventario.

```
--* Incrementar Stock por Devolución
CREATE OR REPLACE TRIGGER trg_incrementar_stock
AFTER INSERT ON devolucion
FOR EACH ROW
BEGIN
    UPDATE producto
    SET stock_actual = stock_actual + :NEW.cantidad
    WHERE id_producto = :NEW.id_producto;

    INSERT INTO inventario(id_producto, tipo_movimiento, cantidad, motivo)
    VALUES (:NEW.id_producto, 'Entrada', :NEW.cantidad,
    'Devolucion automatica - id_devolucion: ' || :NEW.id_devolucion);
END trg_incrementar_stock;
```

```
--* Éxito
SELECT nombre, stock_actual FROM producto WHERE id_producto = 5;
```

Se han recuperado todas las filas: 1 en 0.019 segundos

	NOMBRE	STOCK_ACTUAL
1	Zapato Casual Mocasín	53

```
INSERT INTO devolucion (id_venta, id_producto, cantidad, motivo)
VALUES (1, 5, 1, 'Prueba trigger - empaque roto');

SELECT nombre, stock_actual FROM producto WHERE id_producto = 5;
```

BLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS RESULTADO DE LA CONSULTA

han recuperado todas las filas: 1 en 0.010 segundos

	NOMBRE	STOCK_ACTUAL
1	Zapato Casual Mocasin	54

```
SELECT * FROM inventario WHERE motivo LIKE 'Devolucion automatica%' ORDER BY
id_movimiento DESC;
```

BLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS RESULTADO DE LA CONSULTA ...

han recuperado todas las filas: 2 en 0.020 segundos

	TIPO_MOVIMIENTO	CANTIDAD	FECHA_MOVIMIENTO	MOTIVO
1	Entrada	1	22/07/26	Devolucion automatica - id_devolucio

```
--* Fallo (id_venta que no existe)
INSERT INTO devolucion (id_venta, id_producto, cantidad, motivo)
VALUES (999999, 5, 1, 'Prueba de fallo');
```

BLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS SALIDA DE SCRIPT ...

error que empieza en la línea: 1 del comando :

```
INSERT INTO devolucion (id_venta, id_producto, cantidad, motivo)
VALUES (999999, 5, 1, 'Prueba de fallo')
```

Informe de error -

ORA-02291: restricción de integridad (PROYECTOBD.FK_DEV_VENTA) violada - clave principal no encontrada

8.6. Control de Stock Negativo

Se dispara antes de actualizar el stock_actual de un producto y bloquea la operación si el nuevo valor quedaría negativo.

```
CREATE OR REPLACE TRIGGER trg_control_stock_negativo
BEFORE UPDATE OF stock_actual ON producto
FOR EACH ROW
BEGIN
    IF :NEW.stock_actual < 0 THEN
        RAISE_APPLICATION_ERROR(-20001,
            'Stock negativo no permitido. ' ||
            'Producto: ' || :NEW.nombre ||
            ' | Antes: ' || :OLD.stock_actual ||
            ' | Despues: ' || :NEW.stock_actual);
    END IF;
END trg_control_stock_negativo;
/
```

```
--* Éxito
UPDATE producto SET stock_actual = stock_actual - 1 WHERE id_producto = 1;
SELECT stock_actual FROM producto WHERE id_producto = 1;
```

Problemas SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS RESULTADO DE LA CONSULTA ...

Se han recuperado todas las filas: 1 en 0.018 segundos

	STOCK_ACTUAL
1	58

```
186 --* Fallo
187 UPDATE producto SET stock_actual = -99 WHERE id_producto = 1;
188 SELECT stock_actual FROM producto WHERE id_producto = 1;
189
```

Problemas SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS SALIDA DE SCRIPT ...

Error que empieza en la línea: 1 del comando -
 UPDATE producto SET stock_actual = -99 WHERE id_producto = 1
 Error en la línea de comandos : 1 Columna : 8
 Informe de error -
 Error SQL: ORA-20001: Stock negativo no permitido. Producto: Zapatilla Running Air Max | Antes: 58

8.7. Registro de Cambio de Precio

Se dispara después de actualizar el precio de un producto y guarda en auditoria el precio anterior y el nuevo.

```
CREATE OR REPLACE TRIGGER trg_Registro_Cambio_Precio
AFTER UPDATE OF precio
ON producto
FOR EACH ROW
BEGIN
    IF :NEW.precio < 0 THEN
        RAISE_APPLICATION_ERROR( -20001, 'El precio ingresado no es valido');
    END IF;

    IF NVL(:OLD.precio, 0) <> NVL(:NEW.precio, 0) THEN
        INSERT INTO auditoria (usuario, accion, tabla_afectada, fecha_hora,
            valor_anterior, valor_nuevo)
        VALUES (USER, 'UPDATE', 'PRODUCTO', SYSTIMESTAMP, :OLD.precio, :NEW.precio);
    END IF;
END;
```

```
--* Éxito
UPDATE producto SET precio = 92.50 WHERE id_producto = 1;
SELECT * FROM auditoria WHERE tabla_afectada = 'PRODUCTO' ORDER BY
id_auditoria DESC;
```

Problemas SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS RESULTADO DE LA CONSULTA ...

Se han recuperado todas las filas: 1 en 0.023 segundos

	USUARIO	ACCION	TABLA_AFECTADA	FECHA_HORA	VALOR_ANTERIOR
1	PROYECTOBD	UPDATE	PRODUCTO	22/07/26 04:51:52.884	89.99

```

3  --* Fallo (precio negativo)
4  UPDATE producto SET precio = -10 WHERE id_producto = 1;
5
6  --? 8. Registro de Acceso

```

ror que empieza en la línea: 1 del comando :
 DATE producto SET precio = -10 WHERE id_producto = 1
 forme de error -
 A-02290: restricción de control (PROYECTOBD.SYS_C0010474) violada

8.8. Registro de Acceso

Trigger a nivel de esquema (AFTER LOGON) que registra en auditoria cada inicio de sesión, con usuario y fecha/hora.

```

--? 8. Registro de Acceso
CREATE OR REPLACE TRIGGER trg_registro_acceso
AFTER LOGON ON SCHEMA
BEGIN
  INSERT INTO auditoria (usuario, accion, tabla_afectada, fecha_hora,
    valor_anterior, valor_nuevo)
  VALUES (USER, 'ACCESO', 'BASE DE DATOS', SYSTIMESTAMP, NULL, 'Inicio de
    sesion');
  COMMIT;
END;
/

```

```

SELECT * FROM auditoria
WHERE accion = 'ACCESO'
ORDER BY id_auditoria DESC;

```

recuperado todas las filas: 15 en 0.016 segundos

	ID_AUDITORIA	USUARIO	ACCION	TABLA_AFECTADA	FECHA_HORA
1	130	PROYECTOBD	ACCESO	BASE DE DATOS	22/07/26 04:26:12.714
2	125	PROYECTOBD	ACCESO	BASE DE DATOS	22/07/26 02:43:52.149

9. TRANSACCIONES

9.1. Venta Completa (COMMIT / ROLLBACK)

Agrupar el registro de venta, detalle_venta e inventario en una sola unidad atómica: si algo falla a mitad del proceso, se hace ROLLBACK (o ROLLBACK TO SAVEPOINT) y no queda ninguna venta parcial registrada.

```

DECLARE
    v_id_venta NUMBER;
    v_precio NUMBER;
    v_stock NUMBER;
    v_cantidad NUMBER := 2;
    v_id_prod NUMBER := 1;
    v_subtotal NUMBER;
    v_descuento NUMBER;
    v_iva NUMBER;
    v_total NUMBER;
BEGIN
    SAVEPOINT inicio_venta;

    SELECT precio, stock_actual INTO v_precio, v_stock
    FROM producto WHERE id_producto = v_id_prod;

    IF v_stock < v_cantidad THEN
        RAISE_APPLICATION_ERROR(-20001, 'Stock insuficiente');
    END IF;

    v_subtotal := v_precio * v_cantidad;
    v_descuento := fn_calcular_descuento(v_subtotal, 'Dia del Padre');
    v_iva := fn_calcular_iva(v_subtotal - v_descuento);
    v_total := v_subtotal - v_descuento + v_iva;

```

```

INSERT INTO venta (id_cliente, id_empleado, id_promocion, fecha_venta,
tipo_venta, descuento, iva, total)
VALUES (1, 3, (SELECT id_promocion FROM promocion WHERE nombre = 'Dia del
Padre'),SYSDATE, 'Menor', v_descuento, v_iva, v_total)
RETURNING id_venta INTO v_id_venta;

INSERT INTO detalle_venta (id_venta, id_producto, cantidad,
precio_unitario, subtotal)
VALUES (v_id_venta, v_id_prod, v_cantidad, v_precio, v_subtotal);

COMMIT;
DBMS_OUTPUT.PUT_LINE('Venta registrada, ID: ' || v_id_venta || ' - Total:
' || v_total);
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK TO SAVEPOINT inicio_venta;
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/

```

9.2. Devolución Completa

Agrupar el registro de la devolución y el incremento de stock en inventario como una sola unidad: si falla el ajuste de stock, la devolución tampoco queda registrada.

```

DECLARE
    v_id_devolucion NUMBER;
    v_id_venta NUMBER := 1;
    v_id_prod NUMBER := 14;
    v_cantidad NUMBER := 1;
    v_cantidad_vendida NUMBER;
    v_cantidad_devuelta NUMBER;
BEGIN
    SAVEPOINT inicio_devolucion;

    SELECT cantidad INTO v_cantidad_vendida
    FROM detalle_venta
    WHERE id_venta = v_id_venta AND id_producto = v_id_prod;

    SELECT NVL(SUM(cantidad), 0) INTO v_cantidad_devuelta
    FROM devolucion
    WHERE id_venta = v_id_venta AND id_producto = v_id_prod;

    IF v_cantidad > (v_cantidad_vendida - v_cantidad_devuelta) THEN
        RAISE_APPLICATION_ERROR(-20001, 'No se puede devolver mas de lo
        disponible');
    END IF;

    INSERT INTO devolucion (id_venta, id_producto, cantidad,
    fecha_devolucion, motivo)
    VALUES (v_id_venta, v_id_prod, v_cantidad, SYSDATE, 'Talla incorrecta')
    RETURNING id_devolucion INTO v_id_devolucion;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Devolucion registrada, ID: ' || v_id_devolucion);
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK TO SAVEPOINT inicio_devolucion;
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/

```

9.3. Compra a Proveedor

Agrupar el registro de la compra, su detalle y la entrada correspondiente en inventario, garantizando que el stock y el monto total de la compra queden consistentes entre sí.

```

DECLARE
    v_id_compra NUMBER;
    v_id_prod NUMBER := 5;
    v_cantidad NUMBER := 30;
    vPrecio NUMBER := 28.00;
    v_existe NUMBER;
BEGIN
    SAVEPOINT inicio_compra;

    SELECT COUNT(*) INTO v_existe FROM producto WHERE id_producto = v_id_prod;
    IF v_existe = 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'El producto indicado no existe');
    END IF;

    INSERT INTO compra (id_proveedor, id_employado, fecha_compra, total)
    VALUES (7, 3, SYSDATE, v_cantidad * vPrecio)
    RETURNING id_compra INTO v_id_compra;

    INSERT INTO detalle_compra (id_compra, id_producto, cantidad, precio_unitario, subtotal)
    VALUES (v_id_compra, v_id_prod, v_cantidad, vPrecio, v_cantidad * vPrecio);

    UPDATE producto SET stock_actual = stock_actual + v_cantidad WHERE id_producto = v_id_prod;

    INSERT INTO inventario (id_producto, tipo_movimiento, cantidad, motivo)
    VALUES (v_id_prod, 'Entrada', v_cantidad, 'Compra a proveedor #' || v_id_compra);

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Compra registrada, ID: ' || v_id_compra);
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK TO SAVEPOINT inicio_compra;
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/

```

9.4. Actualización Masiva de Inventario

Actualiza varios productos de inventario en una sola transacción para demostrar las propiedades ACID ante un ajuste masivo de stock.

```
DECLARE
    v_porcentaje NUMBER := 10;
BEGIN
    SAVEPOINT inicio_ajuste;

    INSERT INTO inventario (id_producto, tipo_movimiento, cantidad, motivo)
    SELECT id_producto,
           CASE WHEN v_porcentaje >= 0 THEN 'Entrada' ELSE 'Salida' END,
           ABS(ROUND(stock_actual * (v_porcentaje / 100))),
           'Ajuste masivo de inventario ' || v_porcentaje || '%'
    FROM producto
    WHERE id_categoria = 1
           AND ROUND(stock_actual * (v_porcentaje / 100)) <> 0;

    UPDATE producto
    SET stock_actual = stock_actual + ROUND(stock_actual * (v_porcentaje / 100))
    WHERE id_categoria = 1;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Ajuste masivo aplicado a categoria 1');
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK TO SAVEPOINT inicio_ajuste;
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
```

10. SEGURIDAD - USUARIOS Y PRIVILEGIOS

10.1. Roles creados (CREATE ROLE)

```
--! Roles
CREATE ROLE rol_cajero;
CREATE ROLE rol_vendedor;
CREATE ROLE rol_supervisor_ventas;
CREATE ROLE rol_bodeguero;
CREATE ROLE rol_contador;
CREATE ROLE rol_gerente_sucursal;

GRANT rol_vendedor TO rol_supervisor_ventas;

--! Usuarios
CREATE USER usr_cajero1 IDENTIFIED BY Cajero2026;
CREATE USER usr_vendedor1 IDENTIFIED BY Vendedor2026;
CREATE USER usr_supervisor1 IDENTIFIED BY Supervisor2026;
CREATE USER usr_bodeguero1 IDENTIFIED BY Bodega2026;
CREATE USER usr_contador1 IDENTIFIED BY Conta2026;
CREATE USER usr_gerente1 IDENTIFIED BY Gerente2026;
```

```

--! Conexion
GRANT CONNECT TO usr_cajero1;
GRANT rol_cajero TO usr_cajero1;

GRANT CONNECT TO usr_vendedor1;
GRANT rol_vendedor TO usr_vendedor1;

GRANT CONNECT TO usr_supervisor1;
GRANT rol_supervisor_ventas TO usr_supervisor1;

GRANT CONNECT TO usr_bodeguero1;
GRANT rol_bodeguero TO usr_bodeguero1;

GRANT CONNECT TO usr_contador1;
GRANT rol_contador TO usr_contador1;

GRANT CONNECT TO usr_gerente1;
GRANT rol_gerente_sucursal TO usr_gerente1;

--! Verificacion
SELECT role FROM dba_roles WHERE role LIKE 'ROL_%';
SELECT username, account_status, created FROM dba_users WHERE username LIKE 'USR_%';

```

Se han recuperado todas las filas: 6 en 0.035 segundos

	USERNAME	ACCOUNT_STATUS	CREATED
1	USR_VENDEDOR1	OPEN	17/07/26
2	USR_BODEGUERO1	OPEN	17/07/26
3	USR_GERENTE1	OPEN	17/07/26
4	USR_SUPERVISOR1	OPEN	17/07/26
5	USR_CONTADOR1	OPEN	17/07/26
6	USR_CAJERO1	OPEN	17/07/26

10.2. Usuarios y privilegios asignados por rol

```

--! rol vendedor: registra clientes y ventas
GRANT SELECT ON producto TO rol_vendedor;
GRANT SELECT ON categoria TO rol_vendedor;
GRANT SELECT ON promocion TO rol_vendedor;
GRANT SELECT, INSERT ON cliente TO rol_vendedor;
GRANT SELECT, INSERT ON venta TO rol_vendedor;
GRANT SELECT, INSERT ON detalle_venta TO rol_vendedor;
GRANT SELECT, INSERT ON queja TO rol_vendedor;

GRANT EXECUTE ON sp_registrar_cliente TO rol_vendedor;
GRANT EXECUTE ON sp_registrar_venta TO rol_vendedor;
GRANT EXECUTE ON sp_registrar_queja TO rol_vendedor;
GRANT EXECUTE ON sp_aplicar_promocion TO rol_vendedor;
GRANT EXECUTE ON fn_calcular_iva TO rol_vendedor;
GRANT EXECUTE ON fn_calcular_descuento TO rol_vendedor;
GRANT EXECUTE ON fn_calcular_antiguedad_cliente TO rol_vendedor;
GRANT EXECUTE ON t_producto_venta TO rol_vendedor;
GRANT EXECUTE ON t_lista_productos TO rol_vendedor;

```

```
--! rol cajero: procesa pagos y devoluciones, no crea ventas nuevas
GRANT SELECT ON producto TO rol_cajero;
GRANT SELECT ON promocion TO rol_cajero;
GRANT SELECT ON venta TO rol_cajero;
GRANT SELECT, INSERT ON devolucion TO rol_cajero;

GRANT EXECUTE ON sp_registrar_devolucion TO rol_cajero;
GRANT EXECUTE ON fn_calcular_iva TO rol_cajero;

--! rol bodeguero: stock, movimientos y compras a proveedor
GRANT SELECT, UPDATE ON producto TO rol_bodeguero;
GRANT SELECT, INSERT ON inventario TO rol_bodeguero;
GRANT SELECT ON proveedor TO rol_bodeguero;
GRANT SELECT, INSERT ON compra TO rol_bodeguero;
GRANT SELECT, INSERT ON detalle_compra TO rol_bodeguero;

GRANT EXECUTE ON sp_actualizar_inventario TO rol_bodeguero;
GRANT EXECUTE ON fn_productos_bajo_stock TO rol_bodeguero;

--! rol contador: solo lectura de todo lo financiero
GRANT SELECT ON venta TO rol_contador;
GRANT SELECT ON detalle_venta TO rol_contador;
GRANT SELECT ON compra TO rol_contador;
GRANT SELECT ON detalle_compra TO rol_contador;
GRANT SELECT ON devolucion TO rol_contador;
GRANT SELECT ON promocion TO rol_contador;
GRANT SELECT ON auditoria TO rol_contador;

GRANT EXECUTE ON sp_calcular_ventas_mensuales TO rol_contador;
GRANT EXECUTE ON fn_calcular_comision TO rol_contador;
GRANT EXECUTE ON fn_calcular_iva TO rol_contador;

--! rol supervisor ventas: hereda vendedor + gestiona promociones y catalogo
GRANT SELECT, INSERT, UPDATE ON promocion TO rol_supervisor_ventas;
GRANT SELECT, INSERT, UPDATE ON producto TO rol_supervisor_ventas;
GRANT SELECT, INSERT ON categoria TO rol_supervisor_ventas;
GRANT SELECT ON empleado TO rol_supervisor_ventas;

GRANT EXECUTE ON sp_registrar_producto TO rol_supervisor_ventas;
GRANT EXECUTE ON Clientes_Frecuentes TO rol_supervisor_ventas;
```

```

--! rol gerente sucursal: vision total de la sucursal, gestiona personal y proveedores
GRANT SELECT ON venta TO rol_gerente_sucursal;
GRANT SELECT ON detalle_venta TO rol_gerente_sucursal;
GRANT SELECT ON compra TO rol_gerente_sucursal;
GRANT SELECT ON detalle_compra TO rol_gerente_sucursal;
GRANT SELECT ON producto TO rol_gerente_sucursal;
GRANT SELECT ON inventario TO rol_gerente_sucursal;
GRANT SELECT ON cliente TO rol_gerente_sucursal;
GRANT SELECT, INSERT, UPDATE ON empleado TO rol_gerente_sucursal;
GRANT SELECT, INSERT, UPDATE ON proveedor TO rol_gerente_sucursal;
GRANT SELECT, UPDATE ON queja TO rol_gerente_sucursal;
GRANT SELECT ON auditoria TO rol_gerente_sucursal;

GRANT EXECUTE ON sp_registrar_proveedor TO rol_gerente_sucursal;
GRANT EXECUTE ON sp_calcular_ventas_mensuales TO rol_gerente_sucursal;
GRANT EXECUTE ON fn_calcular_comision TO rol_gerente_sucursal;
GRANT EXECUTE ON Clientes_Frecuentes TO rol_gerente_sucursal;
GRANT EXECUTE ON fn_productos_bajo_stock TO rol_gerente_sucursal;

```

10.3. Justificación de accesos de los roles

Se definieron 6 roles alineados 1 a 1 con los perfiles reales de la empresa, cada uno con acceso únicamente a las tablas y objetos que su función requiere — nunca SELECT ANY TABLE ni privilegios de esquema completo:

- **rol_vendedor:** solo lectura de catálogo (producto, categoria, promocion) y escritura únicamente donde genera datos de negocio (cliente, venta, detalle_venta, queja), a través de los procedimientos sp_registrar_cliente, sp_registrar_venta y sp_registrar_queja. No tiene acceso a empleado, proveedor, compra ni auditoria, porque no le corresponde ver información de personal, proveedores o el historial de auditoría.
- **rol_cajero:** acceso de solo lectura a producto, promocion y venta (para verificar antes de procesar una devolución), y escritura únicamente en devolucion a través de sp_registrar_devolucion. No puede insertar ventas ni modificar productos: su función es procesar pagos y devoluciones, no crear ventas nuevas.
- **rol_bodeguero:** gestiona stock e inventario — lectura y actualización de producto (solo stock_actual, en la práctica), lectura/escritura en inventario, compra y detalle_compra, y lectura de proveedor. No tiene acceso a venta, cliente ni información financiera, porque su función es exclusivamente logística.

- **rol_contador:** acceso de **solo lectura** (ningún INSERT/UPDATE) a las tablas financieras: venta, detalle_venta, compra, detalle_compra, devolucion, promocion y auditoria. Es el único perfil, junto con gerencia, con acceso a auditoria, coherente con su función de control y verificación, no de operación.
- **rol_supervisor_ventas:** hereda rol_vendedor (GRANT rol_vendedor TO rol_supervisor_ventas) y añade permisos de gestión sobre promocion, producto y categoria (INSERT/UPDATE), reflejando que además de vender, puede administrar el catálogo y las campañas — algo que un vendedor raso no necesita.
- **rol_gerente_sucursal:** el perfil con más alcance, pero aun así sin privilegios de administración de base de datos: lectura amplia sobre ventas, compras, inventario, clientes y auditoría (visión total de la sucursal para supervisión), y escritura solo donde le corresponde por su función de gestión de personal y proveedores (empleado, proveedor) y atención de quejas (queja).

En ningún caso un rol operativo tiene privilegios DROP, ALTER, DELETE general o acceso a objetos del sistema, únicamente el usuario propietario del esquema (proyectoBD) tiene control total, y system/sys quedan reservados para tareas administrativas (creación de usuarios, roles y directorios). Esto sigue el principio de menor privilegio: cada usuario de aplicación (usr_cajero1, usr_vendedor1, etc.) solo puede hacer, exactamente, lo que su rol de negocio requiere — y nada más.

11. RESPALDO Y RECUPERACIÓN

Se implementó un respaldo lógico a nivel de esquema utilizando la herramienta Oracle Data Pump ejecutada sobre una instancia de Oracle Database Free 23ai desplegada en un contenedor Podman.

Este tipo de respaldo exporta la estructura completa del esquema (tablas, restricciones, índices, secuencias, vistas, procedimientos, funciones y triggers) junto con todos los datos, generando un archivo binario portable (.dmp) que puede restaurarse en cualquier instancia Oracle compatible.

11.1. Respaldos requeridos

Respaldo	Nombre Backup	Fecha Backup	Hora Backup	Usuario Responsable
Inicial	respaldo_inicial.dmp	12/07/2026	23:53 P.M.	proyectoBD
Intermedio	respaldo_intermedio.dmp	14/07/2026	01:11 A.M.	proyectoBD
Final	respaldo_completo.dmp	22/07/2026	00:07 AM	proyectoBD

12. CONCLUSIONES

- El modelo relacional de 13 tablas permitió representar sin ambigüedades todo el flujo de la empresa de calzado, desde la compra a proveedores hasta la venta, la devolución y el control de inventario, sin duplicar información entre módulos.
- Trasladar reglas de negocio como el cálculo de IVA, el control de stock o la validación de datos de clientes hacia funciones, procedimientos y triggers evita que cada aplicación que consume la base de datos tenga que reimplementar esa lógica por su cuenta, y reduce el riesgo de inconsistencias.
- Las 20 consultas, las 6 vistas y los reportes de desempeño contruidos sobre ellas demuestran que el modelo soporta tanto operaciones transaccionales del día a día como análisis agregados (ventas por mes, productos más vendidos, clientes frecuentes) sin necesidad de herramientas externas.
- El manejo de excepciones con SAVEPOINT en las transacciones críticas (venta, devolución, compra, ajuste masivo) garantiza que un error a mitad de una operación compleja no deje la base de datos en un estado intermedio o inconsistente.
- La combinación de roles con privilegio mínimo y una tabla de auditoría alimentada por triggers da una primera capa de seguridad y trazabilidad razonable para el tamaño del proyecto, sin necesidad de herramientas de auditoría de terceros.
- El respaldo lógico con Oracle Data Pump, ejecutado sobre un contenedor con Oracle Database Free 23ai, resultó una solución práctica para un entorno académico, ya que permite recrear el esquema completo en cualquier máquina que tenga Oracle instalado, sin depender de la infraestructura original.

13. RECOMENDACIONES

- A medida que crezca el volumen de ventas e inventario, conviene revisar los índices existentes y agregar índices sobre columnas que se filtran u ordenan con frecuencia, como fecha_venta o id_cliente en la tabla venta.
- Si el sistema pasa a producción, sería recomendable complementar el respaldo lógico actual con una estrategia de respaldo físico (RMAN) y respaldos incrementales, para reducir el tiempo de recuperación ante una falla mayor.
- Los roles definidos podrían sincronizarse con un sistema de autenticación centralizado (por ejemplo, un directorio LDAP) en caso de que la empresa maneje varias sucursales con múltiples usuarios.
- Sería útil agregar validaciones adicionales en los procedimientos de venta y compra para manejar unidades de medida distintas, en caso de que en el futuro se comercialicen productos que no se vendan por unidad.
- Se recomienda documentar y versionar el script consolidado (DDL, datos, vistas, funciones, procedimientos y triggers) en un repositorio, de manera que cualquier cambio futuro quede registrado y sea fácil de auditar.
- Antes de una eventual puesta en producción, conviene realizar pruebas de carga sobre las consultas más pesadas (las que agregan por mes o calculan promedios) para confirmar que el rendimiento se mantiene con un volumen de datos mucho mayor al usado en las pruebas.

14. BIBLIOGRAFÍA

- Kannan, P., Morin, L., Raphaely, D., Ashdown, L., Baker, D., Carver, D., Chaliha, M., Cheng, B., Day, R., Fogel, S., Llewellyn, B., Lane, P., McDermid, D., Morales, T., Murphy, A., Murray, C., Pelski, S., Rich, K., Romero, A., & Chaudhry, A. (2026). *DBMS_OUTPUT*. Oracle Help Center.
https://docs.oracle.com/en/database/oracle/oracle-database/19/arpls/DBMS_OUTPUT.html
- Moisset, D. (2026). *Descripción: Disparador (OLD y NEW) - Oracle*. TutorialesProgramacionYa.
<https://www.tutorialesprogramacionya.com/oracleya/temarios/descripcion.php?cod=270&punto=1&inicio=>
- Oracle Corporation. (s. f.). *Creación de disparadores*. Oracle Help Center. Recuperado el 22 de julio de 2026, de

<https://docs.oracle.com/es/database/oracle/oracle-database/19/tdddg/creating-triggers.htm>

- Oracle Corporation. (2018). *SAVEPOINT*. Oracle Help Center.
<https://docs.oracle.com/en/database/oracle/oracle-database/18/sqlrf/SAVEPOINT.html>
- Oracle Corporation. (2024). *Oracle Database Free — Get started*.
<https://www.oracle.com/database/free/>
- Oracle Corporation. (2024). *Oracle Database PL/SQL language reference*.
<https://docs.oracle.com/en/database/oracle/oracle-database/>
- Oracle Corporation. (2024). *Oracle Database SQL language reference*.
<https://docs.oracle.com/en/database/oracle/oracle-database/>
- Oracle Corporation. (2024). *Oracle Database utilities — Data Pump*.
<https://docs.oracle.com/en/database/oracle/oracle-database/>
- Oracle Corporation. (2025). *CREATE TYPE*. Oracle Help Center.
<https://docs.oracle.com/en/database/oracle/oracle-database/19/sqlrf/CREATE-TYPE.html>
- Oracle Corporation. (2025). *NVL*. Oracle Help Center.
<https://docs.oracle.com/en/database/oracle/oracle-database/19/sqlrf/NVL.html>
- Oracle Corporation. (2026). *ABS*. Oracle Help Center.
<https://docs.oracle.com/cloud/help/es/epm-common/FRWEB/abs.htm>
- Oracle Corporation. (2026). *PL/SQL control statements*. Oracle Help Center.
<https://docs.oracle.com/en/database/oracle/oracle-database/12.2/lnpls/plsql-control-statements.html>
- TechOnTheNet. (2026). *Oracle / PLSQL: WHEN OTHERS clause*.
https://www.techonthenet.com/oracle/exceptions/when_others.php